

HookSuite

Memoria técnica · Práctica 3 — «Del prototipo al producto»

Máster en Ciberseguridad & IA · Evolve Academy

 **HUECO** — responsable: **José María**. Portada: nº de grupo definitivo, integrantes, fecha de entrega, enlace al repo y enlace al vídeo (apartado 1).

1. Portada

Producto: **HookSuite** — herramienta web de auditoría de seguridad «tipo Burp Suite», con IA. Grupo: **nº por confirmar**. Entrega: **16 de octubre de 2026**.

⚠ **HUECO** — responsable: **José María**. Integrantes con rol, enlace al repositorio público y enlace al vídeo (≥10 min). Se rellena al congelar.

2. Resumen ejecutivo

HookSuite resuelve la auditoría web asistida por IA desde el navegador: interceptar y repetir peticiones, fuzzing, utilidades (hash/encoder/regex), análisis de vulnerabilidades y un módulo de IA que prioriza hallazgos. La P3 parte del prototipo de la P1 y lo lleva a un estado ejecutable, seguro y reproducible.

⚠ **HUECO** — responsable: **José María + Claude**. Cerrar a 1 página cuando el producto esté congelado: qué hace, para quién, y en qué estado final se entrega.

3. Punto de partida

HookSuite nació en la Práctica 1 como prototipo funcional de una suite de auditoría web «tipo Burp Suite» operada desde el navegador y asistida por IA. Quedó desplegado en un servidor Hetzner (www.hooksuite.de) con Docker Compose y, desde la entrega de mayo de 2026, **nadie volvió a tocarlo**: el máster encadenó otras actividades y la Práctica 3 lo retoma tal como quedó.

Qué nos encontramos (verificación en vivo, 29-sep y 4-oct)

- La herramienta **seguía en pie** desde mayo, sin cambios: DNS → Hetzner, nginx/1.31.0, uvicorn y los 36 endpoints de la API respondiendo.
- Núcleo funcional sólido: Spider, Repeater, Intruder, Utilidades (hash/encoder/regex), sesiones por token UUID y WebSocket operativos.
- Dos hallazgos que **reorientan** la Práctica 3 (detalle en el apartado 7):
- La API :8000 estaba **abierta a internet sin ninguna autenticación**, incluidos los endpoints que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales (secretos expuestos).
- El módulo de IA (RF-08), que es lo que da sentido al «con IA» del producto, **nunca llegó a operar** salvo pruebas muy iniciales de la P1: panel construido pero inactivo.

Lo que la P1 nos enseñó a su costa: el saldo de la API

En las primeras pruebas del módulo de IA se cargaron 5 € de saldo en la API y se agotaron en muy poco tiempo. La causa está **reconstruida del historial de git** para esta memoria, no de lo que nadie recordara: el módulo ejecutaba una auditoría completa **en el arranque**, y el servicio está declarado con `restart: unless-stopped`, de modo que cuando el proceso terminaba Docker lo relanzaba y empezaba otra auditoría. Un bucle de auditorías mientras el contenedor estuviera levantado.

Se corrigió el 17 de mayo de 2026 con dos cambios: el módulo pasó a **esperar instrucciones** en lugar de auditar al arrancar —lo que además impide que el proceso termine, y con ello que Docker lo reinicie— y se dejó el análisis en modo simulado por defecto, como freno de mano.

Por qué figura en el punto de partida y no como anécdota: es el único incidente de la P1 con coste económico directo, y la P3 lo hereda a medias. El disparo automático está curado, pero el **tamaño** de una auditoría nunca se acotó (apartados 8 y 10). Es además una de las razones por las que el módulo de IA llegó a la P3 sin haber operado.

Numeración de requisitos

La P1 **no numeraba** los requisitos. El enunciado de la P3 (apartado 3) obliga a numerarlos ahora (RF-xx / RNF-xx) y a usar esa numeración en toda la práctica. La lista de abajo (apartado 4) está **derivada del informe de la P1** —de lo que el producto prometía y hace— y **PENDIENTE DE VALIDAR POR EL GRUPO** en la reunión antes de darla por firme.

Clasificación inicial de requisitos

Usando las cinco categorías del enunciado. El recuento es provisional, a validar por el grupo junto con la numeración.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona; en la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 (12)
Parcial	Existe algo, pero incompleto o con fallos; hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 (3)
Pendiente	Definido, pero sin implementar; se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 (4)
A revisar	El feedback o la experiencia indican que estaba mal planteado; se reformula con justificación	RF-02 · RNF-06 (2)
Descartado	Se decide no implementarlo (debe ser la excepción y justificarse)	— ninguno de momento

⚠ **HUECO** — responsable: **José María**. Feedback docente de la P1: la calificación y los comentarios del equipo evaluador NO constan en el material de la línea. El enunciado los pide expresamente en este apartado. Conseguirlos (José María / portavoz) y resumir aquí qué se nos señaló y cómo lo aborda la P3.

4. Requisitos

Lista completa y numerada. Numeración fijada en la P3 (la P1 no la tenía) a partir del informe de la P1;

PENDIENTE DE VALIDAR POR EL GRUPO. El estado es el de partida (apartado 3); se actualiza a medida que la P3 avanza.

Requisitos funcionales (RF)

ID	Requisito	Estado P1
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	A revisar
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión httpx persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	Cumplido
RF-05	Repeater: modificar y reenviar cualquier petición; respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con *; 4 tipos (SQLi, Blind SQLi, XSS, genérico); resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido
RF-08	Análisis y clasificación de vulnerabilidades con IA (Claude) según OWASP → panel Vulnerabilidades	Parcial
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) → panel Red con código de colores	Pendiente
RF-10	Ejecución de ataques con navegador real (Playwright)	Pendiente
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Pendiente (mejora)

Requisitos no funcionales (RNF)

ID	Requisito	Estado P1
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos {type,payload})	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido

RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	A revisar
RNF-07	Seguridad del propio producto: secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido (a verificar)
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva y el motivo.

RF-02 — proxy del lado servidor (modificado)

- **Versión original (P1):** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) + WebSockets.
- **Versión nueva:** el servidor ejecuta las peticiones directamente con `httpx`, sin interponerse en el navegador del auditor.
- **Motivo:** en la P1, el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a `httpx` directo elimina esa superficie y simplifica el flujo. Es el caso de libro de «requisito reformulado con justificación».
- **Cierre en la Fase 2 (6-oct): el PAC se retiró del código.** Hasta entonces el producto seguía sirviendo el fichero de autoconfiguración aunque el modelo estuviera abandonado desde la P1. Se comprobó que estaba muerto antes de tocarlo: su único consumidor en la interfaz era un componente **que ningún otro fichero importaba**, y el segundo (el lanzador de navegador de RF-09) no está integrado y además apunta a un puerto que la Fase 1 había cerrado, así que ya estaba roto por su cuenta. Se retiraron la ruta del backend, sus dos bloques en Nginx y el componente.
- **Ganancia de seguridad no buscada.** El PAC era la **única** ruta que tenía que quedar sin autenticar, porque un navegador no envía credenciales al pedirlo, y por eso llevaba una excepción expresa en el guardián de la Fase 2. Al jubilarlo, **toda** `/api` exige token sin excepciones: no queda ningún hueco que mantener ni que explicar. Una excepción de seguridad que se puede eliminar vale más que una excepción bien documentada.

RNF-06 — operación degradada (a revisar)

- **Original:** enunciado como principio en la P1, sin demostrar.
- **Nuevo:** la operación degradada se demuestra haciendo que un fallo de la IA sea **distinguible** de un análisis correcto. El clasificador pasa a devolver tres estados donde hoy hay dos: *vulnerable* (con confianza suficiente), *analizado y limpio*, y *no analizado*, este último acompañado del motivo. Encaja con el criterio P3 «se comporta razonablemente ante errores», y se prueba a propósito: con una clave de API inválida el panel debe decir *no analizado*, *no limpio*.
- **Corrección de este documento (8-oct).** Este apartado prometía demostrar RNF-06 «con un umbral sobre el campo `confianza`». Ese umbral **ya existía** en el código desde la P1 —`CONFIDENCE_THRESHOLD = 60`, aplicado en los dos analizadores—, así que la demostración prometida no demostraba nada: se iba a presentar como mejora de la P3 algo ya construido. Se deja escrito en vez de corregirlo en silencio, porque el error está en lo que este documento prometió, no en el producto.
- **Lo que de verdad faltaba, y no estaba en ningún pendiente.** Cuando la IA no puede analizar, el clasificador devuelve `None`, que es **exactamente** lo que devuelve cuando ha analizado y no hay nada: el panel presenta «sin vulnerabilidades» sin distinguir si eso es un resultado o una ignorancia. Y en el camino de fallo de la API es aún más brusco: el cliente lanza la excepción al agotar los reintentos y nadie la captura, de modo que **se interrumpe la fase de ataque completa** en lugar de continuar sin ese análisis. Lo uno no es operación degradada sino silencio, y lo otro no es degradación sino caída.
- **Aplicado y probado el 9-oct (commit 8ebee124).** Los tres estados ya existen en el código: el clasificador devuelve estado: `"analizado"` cuando hay hallazgo, `None` cuando ha analizado y está limpio, y estado: `"no_analizado"` con su motivo cuando no ha podido analizar. Hizo falta añadir además

el módulo `ia/esquemas.py`, que no existía: el cliente reescrito exige un esquema de respuesta y ninguna de las cuatro plantillas de consigna definía uno, de modo que el plan tal como estaba escrito habría fallado en la primera llamada. La prueba automática `ia/tests/test_tres_estados.py` recorre los tres estados en las cuatro vías de análisis con un cliente simulado —sin clave de API ni red, coste cero— y pasa *8 de 8*; sobre el código anterior daba *0 de 8*, y el error era precisamente el descrito arriba: la fase de análisis se caía entera. **Corrección declarada (9-oct, tarde)**. Esta memoria afirmó durante unas horas que lo que impedía ver el estado degradado en el panel era un filtro del orquestador que comparaba la confianza contra `0.6` teniendo la escala en 0-100. Era falso: ese filtro ya estaba corregido desde las 13:03 de ese mismo día (commit `ea317969`, que importa el umbral del clasificador en lugar de duplicarlo en otra escala), cuatro horas antes de escribirse este párrafo. Se deja escrito en vez de corregirse en silencio, porque el modo en que se detectó es parte de lo aprendido. **El motivo real por el que RNF-06 todavía no se ve en el panel es más de fondo**: el estado degradado se calcula, se transporta y se publica dentro del módulo de IA, pero ningún consumidor lo lee —la marca `no_analizado` no aparece ni una sola vez en el frontend ni en el backend—. RNF-06 queda por tanto cumplido *dentro del módulo* y pendiente de *superficie*: lo que falta es el tramo que lleva ese estado hasta la interfaz, y se trata en el apartado 10.

RF-12 — registro y login individual con JWT (mejora, no requisito P1)

- No era requisito de la P1 (venía del road map de mejoras). En la P3 cuenta como **mejora**: suma, pero no compensa un requisito original sin cumplir. Se aborda por su valor de seguridad: es lo que cierra la fuga de datos entre usuarios documentada en los apartados 7 y 8.
- **Corrección de este documento (6-oct)**. Este apartado se titulaba «login individual con JWT», sin la palabra **registro**. El requisito, tal como está escrito, pide las dos cosas: que cada persona **se cree su cuenta** y que entre con ella. Al redactar se estrechó el requisito a la mitad sin declararlo; se deja constancia porque un requisito recortado en silencio es peor que un requisito incumplido y explicado.
- **Cómo se resuelve el registro, y por qué no es abierto**. La persona se da de alta ella misma y elige su propia contraseña —nadie más la conoce, ni quien administra, que es el punto del requisito—, pero necesita un **código de invitación** que reparte el grupo. La razón es el propio producto: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider y el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si faltara, el servicio no arranca, en lugar de arrancar con el registro abierto sin que nadie lo note.
- **Lo que obliga a persistir, y lo que no**. Las **cuentas** se guardan en un volumen, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El **estado de sesión** sigue siendo volátil a propósito, por las razones del apartado 10: son cosas distintas y conviene no confundirlas.

> **Descartados**: ninguno por ahora. El enunciado pide que sean la excepción; si alguno se descarta (candidato: RF-09/DevTools si aprieta el tiempo) se justificará aquí por escrito.

5. Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por `docker-compose`, todos en una red interna `hooksuite-net`, con Nginx como única puerta de entrada. Arquitectura verificada fichero a fichero en la cocina local el 4-oct (`docker-compose.yml` + `infra/nginx.conf`).

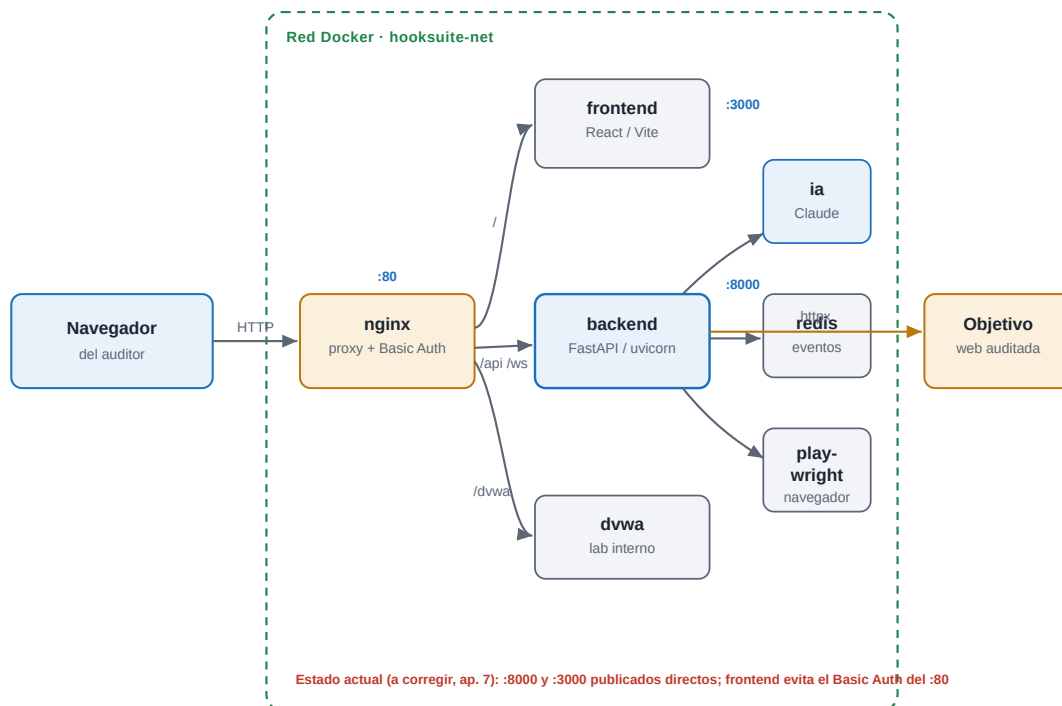


Figura 1: Arquitectura de contenedores de HookSuite: Nginx es la única entrada; el backend FastAPI orquesta Spider/ Repeater/Intruder/IA/Playwright y sale al objetivo auditado con `htpx`. Desde la Fase 2 la autenticación ya no está en Nginx sino en el backend, que exige token en toda `/api` y `/ws/` (apartado 7).

Componentes

Componente	Tecnología	Rol	Puerto (prod.)
nginx	nginx:alpine	Proxy inverso y único punto de entrada; enruta a frontend y backend (el laboratorio salió del proxy en la Fase 1 y el control de acceso pasó al backend en la Fase 2)	80-80
frontend	React + Vite (build propio)	Dashboard web (SPA) servido como estático	3000-80
backend	FastAPI + uvicorn (build propio)	API de auditoría: proxy <code>htpx</code> , Spider, Repeater, Intruder, utilidades, vulnerabilidades y WebSocket	8000-8000
redis	redis:7-alpine	Bus de eventos y estado efímero entre servicios	interno
ia	Python + SDK Anthropic (build propio)	Clasificador de vulnerabilidades con Claude (RF-08)	interno
playwright	Python + Playwright (build propio)	Navegador real para ataques dirigidos (RF-10)	interno
dvwa	vulnerables/web-dvwa	Laboratorio de pruebas, solo accesible por la red interna	interno

Enrutado (Nginx)

Toda petición entra por Nginx, que reparte por ruta (`infra/nginx.conf`):

- `/` → **frontend**: sirve el panel de acceso propio del producto. Hasta la Fase 2 iba protegido con Basic Auth; ya no.
- `/api/` y `/ws/` → **backend** (API y WebSocket). Desde la Fase 2, **ambas exigen token**; sin él responden 401.
- `/check/` y `/proxy.pac` → **retiradas en la Fase 2**, al jubilar el fichero de autoconfiguración de proxy (apartado 4, RF-02).

Desde la Fase 1, Nginx es además la **única** entrada: backend y frontend dejaron de publicar puerto propio, y la ruta hacia el laboratorio vulnerable se retiró del proxy (apartado 7).

El reparto de responsabilidades cambió con la Fase 2, y conviene decirlo porque es una decisión de arquitectura, no un detalle: antes Nginx era quien autenticaba (una contraseña única, igual para todos,

que no distinguía personas) y el backend no pedía nada. Ahora Nginx solo reparte, y **quien autentica y autoriza es el backend**, que es el único que puede saber de quién es cada dato.

Dos entornos: la cocina y la caja

Toda la práctica se trabajó sobre **dos** instalaciones de HookSuite, y la separación entre ellas es una decisión de ingeniería, no una comodidad. Conviene explicarla antes de los apartados 6, 7 y 8, porque es el marco en el que ocurre todo lo que esos apartados cuentan.

Entorno	Para qué	Quién lo toca
La cocina (servidor doméstico propio)	Donde se reproduce cada defecto, se escribe el arreglo, se reconstruyen los contenedores y se verifica que funciona. Todas las pruebas de las Fases 1 y 2 se ejecutaron aquí.	Cualquier cambio, cuantas veces haga falta
La caja (servidor público contratado)	La instalación que usan las personas. Es la herramienta final.	Solo recibe despliegues. Ni un comando de prueba, ni un arreglo rápido, ni una reconstrucción

El orden de trabajo, siempre el mismo. Reproducir el fallo en la cocina → escribir el arreglo → reconstruir y verificar en la cocina → y solo entonces desplegar a la caja y volver a verificar allí. Ninguna de las dos verificaciones sustituye a la otra, y la memoria las mantiene **separadas a propósito**: en la matriz del apartado 9 hay requisitos marcados como «verificado en cocina, pendiente de producción», y el apartado 7 estuvo diciendo expresamente que no podía afirmar nada de la instalación pública hasta que el despliegue se hizo de verdad.

El despliegue descarta lo que encuentre en la caja. Se trae el código con `git fetch` y `git reset --hard`, nunca con `git pull`. La diferencia importa: `reset --hard` **borra** cualquier cambio local, así que si alguien editara un fichero en la caja, el siguiente despliegue se lo llevaría. Eso no es un efecto secundario que se tolere, es el comportamiento que se busca: convierte la caja en un destino y no en un sitio donde trabajar.

Y la regla nació de un fallo propio, que es la razón de que sea tan estricta. La interfaz que corría en producción se había reescrito directamente en la caja y **no estaba en ninguna rama del repositorio**: existía solo allí, sin committear, sin revisar y sin forma de reconstruirla si la máquina se perdía. Recuperarla fue una de las tareas de la Fase 0. La prohibición de editar en la caja no es prudencia abstracta: es la respuesta a haber estado a un fallo de disco de perder el frontend.

Por qué en esta herramienta pesa más que en otras. HookSuite lanza tráfico contra sistemas de terceros —rastrea, repite peticiones y prueba cargas—. Ensayar eso en la instalación que otras personas están usando no arriesga solo nuestros datos: arriesga dirigir tráfico de ataque desde la infraestructura pública del grupo sin querer. El laboratorio vulnerable con el que se prueba vive, por eso, en la red interna de la cocina (apartado 7).

Lo que esta separación NO garantiza, dicho sin adornos. La cocina no es idéntica a producción: comparte máquina con otros servicios y remapea puertos (ver la nota de reproducibilidad al final de este apartado). Por tanto «verificado en la cocina» es una afirmación más débil que «verificado en producción», y la memoria no las confunde en ningún sitio. El caso que lo demuestra se cuenta en el apartado 8: un despliegue que habría terminado sin errores **sin aplicar** la configuración del proxy, porque nombrar un servicio en un comando no prueba que el comando lo recree.

Decisiones técnicas

El bus interno entre el backend y el modulo de IA (RF-08)

El control de acceso de la Fase 2 tuvo una consecuencia que no se previó: **dejo al propio modulo de IA fuera**. Ese modulo preguntaba al backend por HTTP si habia trabajo pendiente, y esa ruta quedo protegida como todas las demas, de modo que desde el 6 de octubre recibia un rechazo por falta de credencial. El apartado 8 cuenta como se descubrio y por que nadie lo noto.

Reconectarlo admitia tres caminos, y la eleccion dice mas del producto que la implementacion:

1. **Darle una credencial de servicio** al modulo y que el control la aceptara. Es el camino corto, y abre una excepcion precisamente en la pieza cuyo valor es no tenerlas: el control se aplica por grupo de rutas para que una ruta nueva nazca protegida, y lo que hay que acordarse de poner es lo que se olvida.
2. **Dejar esa ruta sin control.** Peor: vive bajo el prefijo que el proxy publica a internet, asi que cualquiera podria encolar una auditoria **contra el objetivo que quisiera** desde nuestra infraestructura. Es el riesgo exacto que el codigo de invitacion del registro existe para evitar (apartado 7).
3. **Un bus interno**, que es lo que se hizo. El despliegue ya incluye un bus de eventos y el backend ya lo consumia para otro flujo. El panel pide la auditoria por la ruta normal —con su credencial, como cualquier otra peticion—, el backend publica la orden en el bus, y el modulo de IA la recoge de ahi. El salto interno no necesita autenticacion porque **la frontera la pone la red**: ese bus no esta publicado. Y no hay credencial nueva que custodiar, rotar ni documentar.

El detalle que ordena todo lo demas: el dueño viaja con la orden. Cada mensaje lleva el espacio de datos de quien pidio la auditoria, tomado del control de acceso en el momento de publicar. El modulo de IA construye su trabajo con ese dato y lo devuelve tal cual, de modo que los hallazgos aterrizan en la sesion de quien los pidio. Eso resuelve de raiz un problema que arrastrabamos: el modulo usaba un identificador de sesion **fijo**, heredado de antes del sistema de usuarios, que no era de nadie. No habia que arbitrar entre cuatro identificadores: habia que hacer que el dato viajara.

Y la contrapartida de un bus, que es la opuesta a la de un control de acceso. Un control de acceso falla cerrando; un bus falla **abriendo**: un mensaje que llega sin dueño podria repartirse «por si acaso», y eso es literalmente la fuga que cerro la Fase 2. Asi que un mensaje sin dueño **se descarta**, y hay pruebas de eso y de que dos auditores no ven lo del otro (apartado 8).

Proxy del lado servidor: pivote PAC → httpx (RF-02)

La decisión arquitectónica de más peso heredada de la P1. En origen, HookSuite interceptaba el tráfico del navegador con un archivo de autoconfiguración (PAC) + WebSockets; ese modelo quedó expuesto y se saturó con tráfico de bots. Se pivotó a que **el servidor ejecute las peticiones directamente con httpx**: menos superficie y un flujo más simple, a cambio de perder la captura pasiva del tráfico del navegador —justo lo que retoman después DevTools (RF-09) y Playwright (RF-10)—. Versión original/nueva/justificación detalladas en el apartado 4.

Frontend: reescritura a Tailwind (cambio respecto a la P1)

La interfaz que corría en producción (www.hooksuite.de) se había reescrito al framework de estilos **Tailwind**, abandonando el sistema de variables CSS propio de la P1. Esa versión —la desplegada y en uso— es la que se toma como **base del frontend en la P3**. En la Fase 0 se reconcilió al repositorio (vivía solo en la caja, sin commitear); el detalle del proceso está en el apartado 8.

Puntos a corregir, detectados al montar la cocina

- **CORS abierto a cualquier origen** en el backend (FastAPI) → **corregido en la Fase 1**: lista cerrada de orígenes, configurable por entorno (apartado 7).
- **Verificación SSL desactivada** en el cliente httpx: justificable en una herramienta de auditoría (objetivos con certificados autofirmados), pero se documenta expresamente como decisión, no como descuido.
- **Acoplamiento Nginx ↔ DVWA**: `infra/nginx.conf` hacía del laboratorio un requisito de arranque del proxy (host not found in upstream) → **resuelto en la Fase 1** al retirar su ruta del proxy: el laboratorio queda solo en la red interna y Nginx ya no depende de él.
- **Playwright fuera de la red**: en el `docker-compose.yml` el servicio playwright no declaraba `networks`: `hooksuite-net`, así que quedaba aislado y no hablaba con el backend ni con el laboratorio, pese a tener configurados `BACKEND_URL=http://backend:8000` y `DVWA_URL=http://dvwa:80` (causa raíz de RF-10/RF-08 sin operar) → **corregido el 9-oct** con una línea en el `compose`, verificado con `docker compose config`, que ahora sitúa los siete servicios en `hooksuite-net`. **Corrección declarada (R9)**: hasta ese arreglo la frase «todos en una red interna hooksuite-net» del apartado 5 era inexacta y el documento se contradecía a sí mismo; se deja dicho aquí en vez de reescribirlo en silencio.

> **Nota de reproducibilidad (cocina).** En el mediaserver los puertos 80/3000/8000 ya los ocupan otros servicios, así que la cocina los remapea a 8880/8830/8800 mediante un override local que no toca el compose versionado. Es solo del entorno de pruebas; la arquitectura de producción es la de la tabla.

⚠ **HUECO** — responsable: **José María + Claude**. Añadir, si procede, un diagrama de secuencia del ciclo IA ↔ Playwright ↔ Backend cuando RF-08/RF-10 estén operativos.

6. Funcionalidades implementadas

Cada funcionalidad con capturas: qué hace, cómo se usa y qué requisitos cubre.

- Proxy interceptor, Repeater, Intruder, Utilidades (hash/encoder/regex), Vulnerabilidades, Red/DevTools, módulo IA.

La auditoría con IA, por fin arrancable desde el panel (RF-08)

Hasta el 10 de octubre el modulo de IA **no se podía usar**: existía, esperaba ordenes y no habia forma de darselas —ni un boton en ninguna pantalla—. Ahora la pantalla de vulnerabilidades lleva un campo de objetivo y un boton que lanza la auditoria. El objetivo por omision es el laboratorio interno, que es nuestro y no sale de la red interna.

Dos detalles que no son adorno:

- **Encolar no es atender.** Si el modulo de IA no esta en marcha, el backend lo detecta —nadie escucha el bus— y la pantalla lo dice en rojo. Callarlo dejaria al operador esperando un resultado que no va a llegar, que es el mismo fallo silencioso que este documento persigue en otros tres sitios.
- **El estado degradado se ve.** Lo que la IA no pudo analizar aparece **arriba y en ambar**, separado de los hallazgos, con su motivo. Va separado a proposito: un fallo de analisis no es una vulnerabilidad, y mezclarlos obligaria a inventarle severidad y confianza a algo que precisamente no se pudo valorar. Y el mensaje de «no se han detectado vulnerabilidades» deja de aparecer cuando hay analisis incompletos: en su lugar dice que no hay hallazgos **pero que hay analisis sin completar**. Esa frase era, literalmente, el unico sitio donde el producto podia mentirle al operador sin que nada fallara.

Defectos corregidos en la Fase 1

Los **doce** defectos del prototipo que recoge la tabla, todos reproducidos antes de corregirlos y verificados después en producción. El detalle de cada uno (causa raíz, prueba y autoría) está en el diario del repositorio.

> **Corrección declarada (R9).** Hasta el 10-oct esta frase decía «diez defectos» y la tabla listaba doce. No fue un desfase posterior: el recuento y la tabla se escribieron en el mismo cambio del 5-oct, así que la cifra estuvo mal desde el principio y se publicó así. Se corrige aquí en vez de cambiarla en silencio. El número va ahora **atado a la tabla** —«los doce que recoge la tabla»— para que no pueda volver a divergir de ella.

Qué fallaba	Causa	Requisito
El Spider entraba en bucle y martilleaba el objetivo	No canonicalizaba las direcciones: el mismo recurso generaba variantes infinitas	RF-03
El Spider podía dirigirse a servicios internos del propio servidor	Faltaba filtrar destinos no legítimos	RNF-07
El botón de detener el Spider no existía en la interfaz	El servidor aceptaba la orden, pero la pantalla no ofrecía forma de darla	RF-03
Detener el Spider no detenía nada	El bucle de rastreo no consultaba la señal de parada	RF-03
Un mismo formulario aparecía decenas de veces	No se agrupaban: se emitía uno por página visitada	RF-03
Cancelar el Intruder devolvía un error del servidor	La operación invocaba un método inexistente	RF-06
Dos operaciones del proxy devolvían error del servidor	Un error de sintaxis construía un tipo de dato equivocado	RF-02
Importar un comando curl rompía las direcciones seguras	Una expresión regular mal escapada recortaba el esquema	RF-05
Los avisos de error no decían qué había fallado	Se mostraba el texto crudo de la librería de red	RF-05

Dos pantallas no recibían avisos en tiempo real	El servidor guardaba una sola conexión por sesión y la segunda desplazaba a la primera	RNF-04
El agente del cortafuegos era manipulable por cualquiera	Su canal se creaba con permisos para todo el sistema y no validaba lo recibido	RNF-08
El estado de sesión podía crecer sin límite	El recolector existía pero nunca se ejecutaba, y cualquier identificador nuevo creaba una sesión	RNF-07

Sistema de acceso por usuario (Fase 2)

Funcionalidad nueva, no un defecto corregido: cada persona se crea su cuenta, entra con ella y **solo ve sus propias auditorías**. Cubre RF-12 y es lo que cierra las cuatro fugas del apartado 7.

- **Registro con código de invitación.** La persona elige su contraseña y nadie más la conoce —ni quien administra—, pero necesita un código que reparte el grupo. No es abierto a propósito: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider o el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si falta, **el servicio no arranca**, en vez de arrancar con el registro abierto sin que nadie lo note.
- **Entrada con token firmado por el servidor.** Las contraseñas se guardan como hash, nunca en claro. El algoritmo de firma está fijado en el servidor y no se lee del propio token, porque aceptarlo del token permite presentar uno sin firma y que el servidor lo dé por bueno.
- **Cada usuario, su espacio.** El historial, las cookies capturadas y los hallazgos cuelgan de la cuenta de quien los generó. El identificador de ese espacio **sale siempre del token**: ni de la URL ni del cuerpo de la petición, para que el cliente no pueda elegir dónde escribe.
- **Cuentas persistentes.** Se guardan en un volumen propio, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El estado de sesión, en cambio, sigue siendo volátil a propósito (apartado 10): son cosas distintas.

Nota sobre el último: no era el defecto que teníamos anotado. El pendiente decía «las sesiones no se conservan al reiniciar»; al abrirlo resultó que el problema real era el contrario y más grave —podían crecer sin límite desde fuera—. La volatilidad se justifica en el apartado 10; lo que se corrigió fue el crecimiento.

⚠ **HUECO** — responsable: **José María + Claude**. Documentar cada funcionalidad (qué hace, cómo se usa, qué requisitos cubre) con capturas de producto; las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. El commit de cada módulo se firma a nombre de su responsable cuando toque (R2): Ivan/frontend, Macarena/backend, Nacho/playwright, Carlos/DevTools, José María/IA.

7. Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. (Base en REQUISITOS.md §3.) Contenido ya verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Verificado el 4-oct: **las cuatro están muertas** (la API responde 401 / "API key is invalid."). Aun así, el enunciado (§7) obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo.

- El 4-oct se **reescribió el historial** con `git-filter-repo`: se redactaron las claves por patrón y se eliminaron los `.env` rastreados y la basura (`node_modules`, `venv` de Windows), conservando el código, las 7 ramas y el `.env.example` (con la clave ya redactada).
- **Ejecutado en GitHub el 4-oct** (force-push de las 7 ramas, con copia de seguridad previa). Verificado sobre un clon nuevo: **0 claves en todo el historial** de las ramas. El repositorio público queda sin secretos.
- Salvedad: GitHub conserva un tiempo los refs internos de los **pull requests** antiguos apuntando a los commits viejos; los purga por su cuenta. Como las claves están muertas, no supone riesgo.

LIMPIEZA EJECUTADA Y VERIFICADA

Exposición de la caja: cómo estaba

Estado de partida, verificado en vivo el 29-sep y el 4-oct (evidencia fotográfica en el apartado 8):

- La API :8000 **no tenía autenticación**: lo declaraba su propia `openapi.json` (40 operaciones sin seguridad) y `/health` respondía sin credenciales.
- El Basic Auth del :80 se **evitaba** sirviendo el frontend directamente por :3000.
- El servidor **no tenía cortafuegos** (política de aceptación por defecto) ni protección contra fuerza bruta, y acumulaba unos **127.000 intentos de acceso en 7 días**.
- DVWA **no** estaba expuesto a internet (solo red interna) — corrige un supuesto previo nuestro.

Endurecimiento aplicado en la Fase 1 (5-oct)

Cierre de la exposición

La API y el frontend dejaron de publicar puerto propio: **la única entrada es el proxy**. No bastaba con retirarlos del `docker-compose.yml`, porque el frontend llevaba la dirección del servidor incrustada en tiempo de compilación y hablaba directamente con la API; hubo que **reconstruirlo para que use el mismo origen** y que sus llamadas (incluido el WebSocket) pasen por el proxy. Comprobado desde fuera tras el despliegue: los puertos directos **rechazan la conexión**, el proxy responde, y el paquete del frontend ya no contiene ninguna referencia a la dirección antigua.

Orígenes permitidos (CORS)

El backend aceptaba **cualquier** origen con credenciales, combinación además inválida por especificación: en la práctica, el servidor **reflejaba el origen de quien preguntara**, de modo que cualquier web podía hacerle peticiones autenticadas. Se sustituyó por una **lista cerrada** configurable por entorno. Comprobado: un origen arbitrario ya no recibe permiso; uno legítimo sí.

El laboratorio vulnerable, solo interno

Se retiró del proxy la ruta que lo publicaba. Queda accesible únicamente desde la red interna, para las pruebas del propio producto.

Endurecimiento del servidor

- **Cortafuegos** activo, con política de denegación por defecto y únicamente el acceso remoto y el proxy permitidos.
- **Bloqueo automático de fuerza bruta** contra el acceso remoto: en su primer minuto ya registraba 148 intentos fallidos y había bloqueado dos direcciones.
- **Acceso remoto solo por clave**: se desactivó la autenticación por contraseña. La prueba más limpia es el propio mensaje del servidor, que pasó de ofrecer «clave o contraseña» a admitir **solo clave**.
- **Limpieza de reglas abandonadas**: siete reglas heredadas de la P1 —una de ellas con la dirección pública de un integrante y otra con un valor de ejemplo— se retiraron tras comprobar que ningún servicio escuchaba en esos puertos.
- **Agente de firewall duplicado**: se encontró un segundo proceso del agente, lanzado a mano en mayo y fuera del gestor de servicios, con permisos de administración del cortafuegos. Se terminó; queda uno solo, gestionado por el sistema.
- **Sistema operativo**: se estrenó el núcleo ya instalado pero sin aplicar desde mayo. Antes de reiniciar se corrigió una condición de carrera en el arranque que habría dejado el agente del cortafuegos en bucle (ver apartado 8).

Dos matices honestos, para no atribuirnos de más

- **El acceso de administrador por contraseña ya estaba bloqueado** por la configuración por defecto del sistema: los 127.000 intentos registrados nunca tuvieron ninguna posibilidad. Lo que se cerró fue la autenticación por contraseña **a nivel general**. Se hizo igual porque deja de depender de un valor por defecto que una actualización podría cambiar, y porque cierra la puerta a cualquier usuario futuro.
- **El cortafuegos del sistema no filtra los puertos que publica Docker**, cuyas reglas se evalúan antes. Quien cerró de verdad la API y el frontend fue el despliegue, al dejar de publicarlos. El cortafuegos protege el resto del servidor, no esos puertos.

EXPOSICIÓN CERRADA Y SERVIDOR ENDURECIDO

Autenticación y aislamiento: de cuatro fugas a cero (Fase 2)

Hasta la Fase 2, la API y el WebSocket **no tenían autenticación** por detrás del proxy: el único control era el Basic Auth de la entrada, que además no distinguía personas. Eso no era una carencia, sino **cuatro fallos distintos**, los cuatro medidos en la cocina el 6-oct antes de tocar una línea de código. La tabla enfrenta cada uno con lo que lo cierra; la evidencia de los dos lados está en el apartado 8.

Cómo estaba (medido el 6-oct)	Cómo queda (Fase 2)
La API entera respondía sin credencial. La asimetría lo delataba: GET /health devolvía 401 por el Basic Auth de la entrada, pero GET /api/session/new devolvía 200 y entregaba un token. El proxy protegía unas rutas y dejaba /api abierta.	Un guardián aplicado por router , no ruta por ruta, para que una ruta nueva nazca protegida. Toda /api y /ws/ exigen token: sin él, 401 con cabecera WWW-Authenticate. El Basic Auth de la entrada se retiró, porque la credencial la pide ahora quien debe.
El token de sesión no era una credencial, era un nombre. El backend creaba sesión para cualquier cadena inventada, así que no había frontera que violar: bastaba escribir un token cualquiera en la URL.	El token lo firma el servidor y el espacio de datos se deduce del token, nunca de la URL ni del cuerpo. El navegador dejó de inventarse su identificador. La ruta que repartía identificadores sin valor de credencial se retiró.
La cookie de sesión de la web auditada se compartía entre todos. Se guardaba en una tabla global indexada solo por dominio, sin dueño; un segundo usuario leía la cookie capturada por el primero, y se obtenía además sin token.	Las cookies viven dentro de la sesión de cada usuario y la tabla global desapareció. Verificado: el segundo usuario ya no obtiene la cookie del primero.
Los hallazgos se difundían a todos los paneles. Una vulnerabilidad publicada sin token aparecía en la sesión de dos auditores distintos: en una herramienta de auditoría, eso es una fuga de datos del cliente de otro.	Los dos endpoints que admitían peticiones sin token escriben ahora solo en el espacio de quien llama. Verificado: el segundo usuario no ve el hallazgo del primero.

Un quinto fallo, que introdujo la propia corrección

Se documenta porque el enunciado pide las pruebas que fallaron, y porque la lección vale más que el acierto. El guardián comprobaba que el espacio de datos nombrado **en la ruta** fuera el tuyo, pero **cinco rutas reciben ese nombre en el cuerpo de la petición**, y ahí no miraba nadie: un usuario autenticado podía escribir en el espacio de otro con solo ponerlo en el cuerpo. Es el mismo error que este documento le reprocha al código anterior —dejar que el cliente elija dónde escribe— un nivel más abajo.

La peor de las cinco fue el Repeater, y se escapó del primer inventario por buscarla con el nombre de modelo equivocado. Ahí el daño no era ensuciar el historial ajeno: el Repeater mantiene **un cliente HTTP persistente por usuario** que acumula las cookies del objetivo auditado —el mecanismo que da servicio a RF-04—, de modo que nombrar a otro en el cuerpo reutilizaba **su sesión ya autenticada contra la web auditada**.

Se descartó la corrección evidente, que era validar ese campo contra el token: funciona, pero deja el dato en manos del cliente y obliga a acordarse en cada modelo nuevo. Las rutas toman el espacio del token y **descartan** lo que venga en el cuerpo: lo que no se lee no se puede falsear. Comprobado como ataque real (apartado 8).

Despliegado y verificado en producción (8-oct)

Hasta esta fecha, este apartado decía expresamente que no podía afirmar nada de la instalación pública: todo estaba **verificado en la cocina**, y la caja seguía sirviendo la Fase 1 con las cuatro fugas abiertas. Se dejó escrito así a propósito, porque la tentación de contar una mejora en presente antes de que llegue a producción es justo como una memoria acaba describiendo un producto que no existe. El 7 de octubre se comprobó además **en vivo y en la propia instalación pública** que las fugas no eran teóricas: la API respondía sin credencial a cualquiera.

El despliegue se hizo el **8 de octubre a las 06:52 UTC**, en cuatro pasos con punto de retorno previo: se etiquetaron las imágenes en servicio y se copió la configuración, se mergeó el cambio a la rama principal, se trajo el código a la caja con `git reset --hard` y se generaron los secretos, y se reconstruyeron los contenedores. La verificación inmediata, con el sistema ya en marcha:

Qué se midió	Resultado
Entrada del panel (/)	200 — carga sin pedir Basic Auth : la contraseña compartida está retirada

API (/api/auth/yo)	401 — exige token ; la fuga nº 1 de la columna izquierda, cerrada
Reinicios del backend	0 — prueba que los secretos llegaron: el código se niega a arrancar sin ellos
Contenedores recreados	backend, frontend y nginx; los otros cuatro, intactos a propósito

El dato de los **cero reinicios** no es decorativo: el acceso por usuario se diseñó para **no arrancar** si faltan el secreto de firma o el código de invitación, en lugar de arrancar sin autenticación pareciendo correcto. Que el contenedor esté sirviendo sin un solo reinicio es la prueba de que esa comprobación se superó.

El flujo completo del acceso, capturado en la instalación pública el mismo día del despliegue:

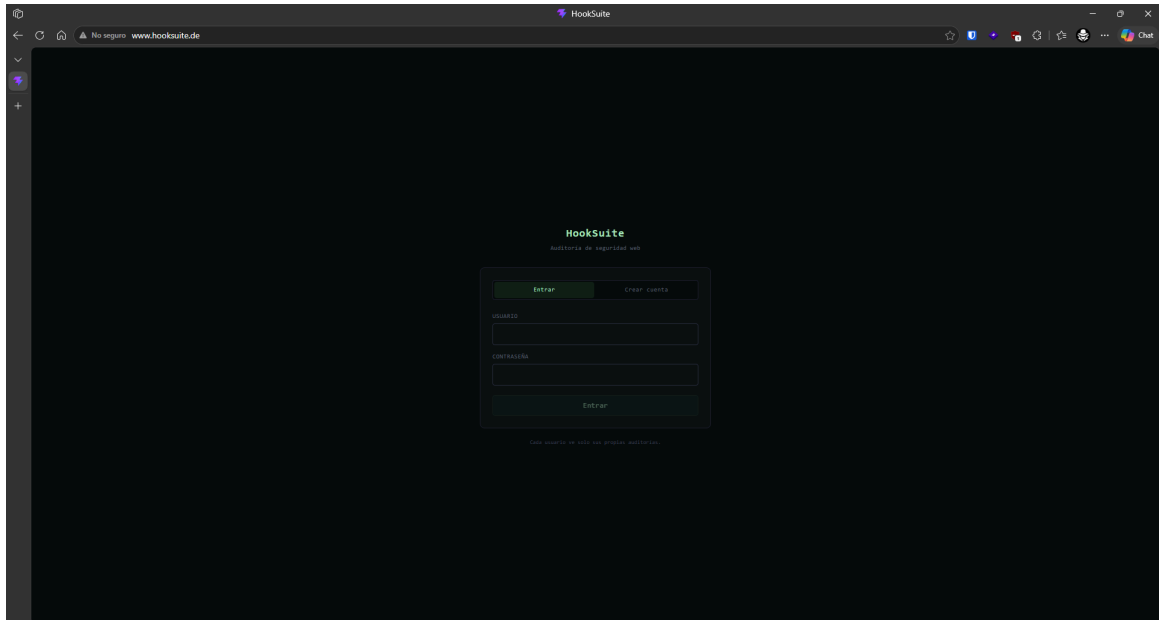


Figura 2: RF-12 — la entrada de `www.hooksuite.de` carga **sin** el cuadro de Basic Auth que antes pedía una contraseña compartida por todo el grupo. La puerta la guarda ahora el acceso por usuario.

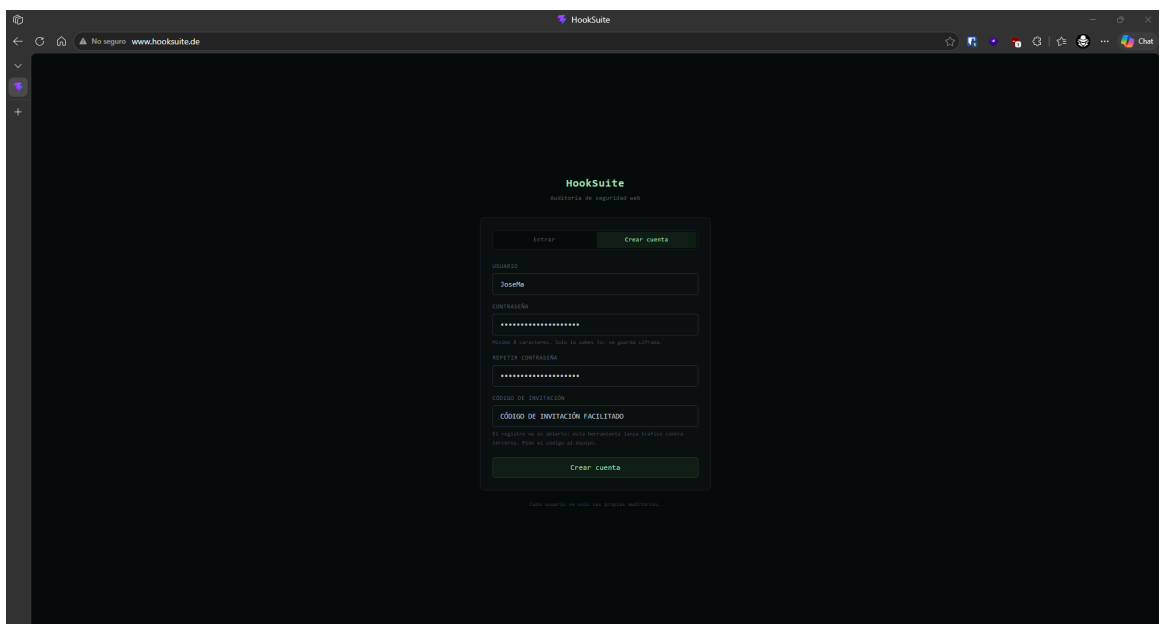


Figura 3: RF-12 — el alta exige un **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, y con altas anónimas cualquiera atacaría a quien quisiera desde nuestra infraestructura. El campo de la imagen muestra el texto de ayuda, no el código.

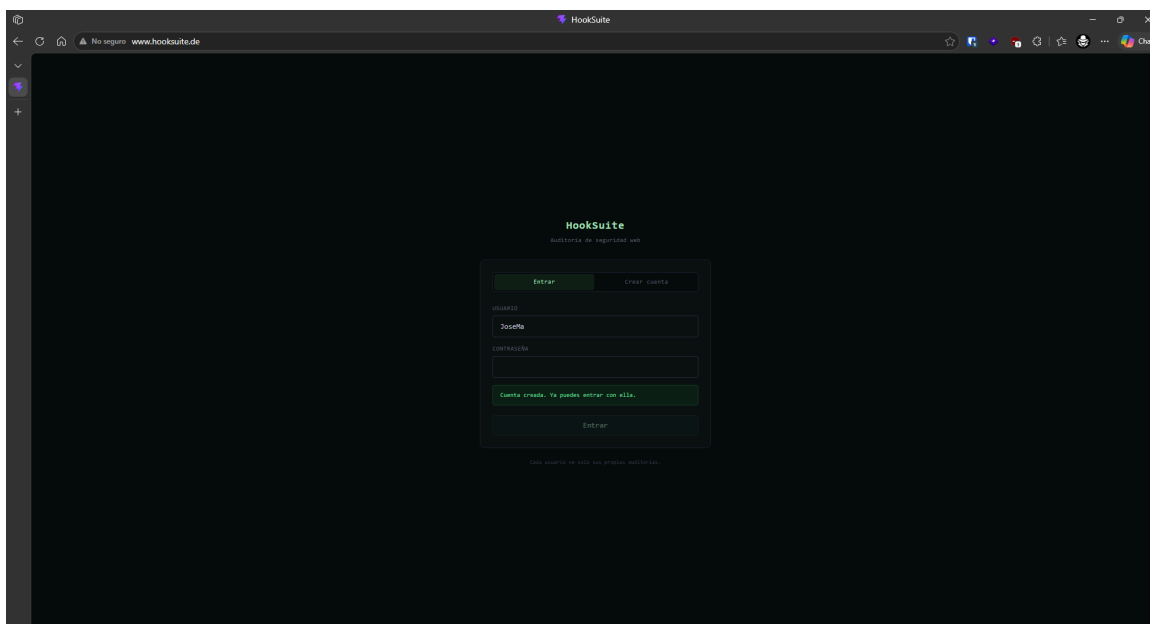


Figura 4: RF-12 — confirmación del alta del primer usuario de producción, creado con el código de invitación. La instalación arrancó deliberadamente con cero usuarios.

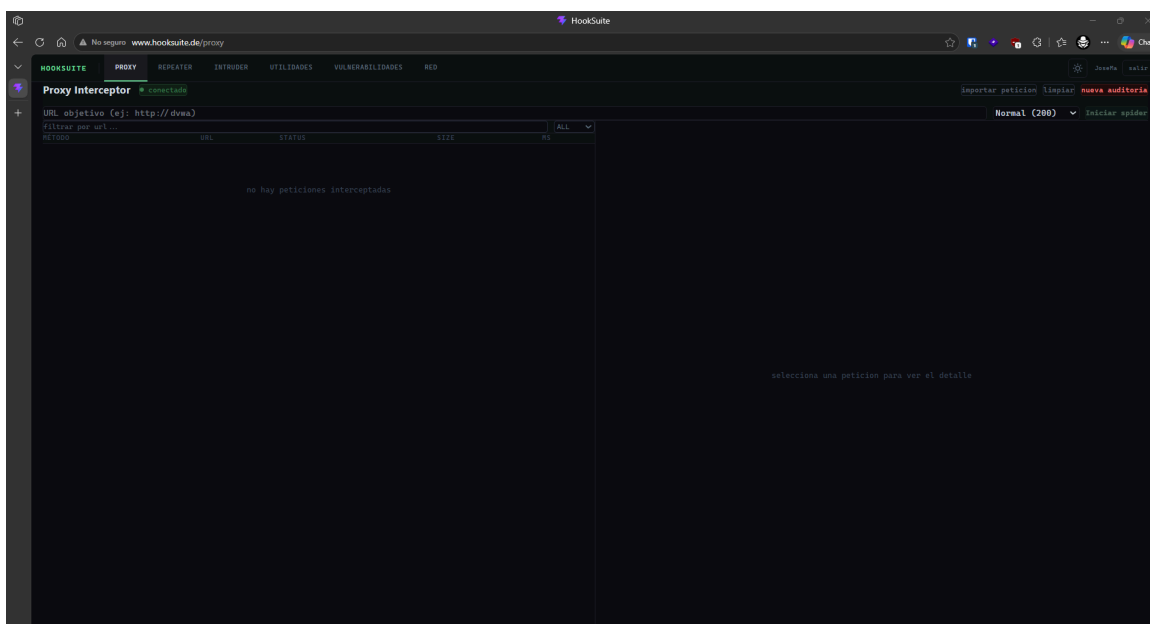


Figura 5: RF-12 y RNF-07 — sesión iniciada: el usuario figura en la cabecera y el interceptor aparece como **conectado**, lo que prueba que el WebSocket viaja también autenticado y no solo la API.

Lo que sigue sin poder afirmarse, y no se disfraza. El dominio publica también una dirección IPv6, y por ese camino la comprobación devolvió 404 en lugar del panel. No está confirmado qué ve un visitante real que llegue por IPv6, porque desde el equipo de pruebas no hay salida por esa vía. Mientras no se compruebe desde una red con IPv6, la disponibilidad solo está demostrada por IPv4.

CUATRO FUGAS CERRADAS Y VERIFICADAS EN PRODUCCIÓN (8-oct, 06:52 UTC) · PENDIENTE: COMPROBAR LA LLEGADA POR IPv6

Lo que sigue sin cubrir

- **Sin TLS** (RNF-09): el acceso viaja en claro, así que el token de sesión es interceptable por quien esté en el camino. Pendiente de configurar en el proxy.
- **Credencial de servicio para los módulos internos** (Fase 3): el clasificador de IA y el agente de Playwright publican sus hallazgos en rutas que ahora exigen token, y no tienen credencial. Comprobado en la instalación de producción el 7-oct: los dos contenedores **sí están en marcha**, pero ninguno llega hoy a esas rutas — el clasificador está conectado al servidor y a la espera de instrucciones, que no

llegan porque RF-08 todavía no tiene disparador, y el agente de Playwright no alcanza siquiera el servidor por el fallo de red descrito en el apartado 5. Es decir: el acceso por usuario no degrada nada que hoy funcione, pero en cuanto RF-08 tenga su disparador el clasificador recibirá 401 si no se le da credencial de servicio.

> **Corrección declarada de este documento (R9).** Una versión anterior de este apartado justificaba lo anterior diciendo que ninguno de los dos módulos estaba en marcha. Era falso: valía para el entorno de pruebas, donde no se levantan, y se escribió sin comprobar la instalación de producción. La conclusión se sostiene, pero por un motivo distinto del que se dio, y la diferencia importa: uno de los dos está vivo y conectado al servidor.

- **Modelo STRIDE, validación de entradas, dependencias y datos personales**, abajo.

⚠ HUECO — responsable: **José María**. Completar el modelo STRIDE, validación de entradas, dependencias y tratamiento de datos personales.

8. Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. El detalle caso a caso (PR-01...PR-20) se mantiene en la matriz del apartado 9 y en el diario del repo.

Dónde se tomó cada evidencia, porque cambia lo que vale. Salvo cuando se dice lo contrario, **toda** la verificación de las Fases 1 y 2 se ejecutó en la cocina —el entorno de pruebas descrito en el apartado 5—, y se repitió después contra la instalación pública tras cada despliegue. Las dos se registran por separado y con su fecha: una prueba en la cocina dice que el arreglo funciona; solo la de producción dice que **está puesto**. Esa distinción no es formalismo: más abajo se cuenta un despliegue que habría pasado por correcto sin aplicar la configuración que se le pedía.

Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct

La prueba PR-16 —llamar a la API sin credenciales— y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Capturas obtenidas antes de cualquier arreglo (R6: la evidencia de un fallo se captura antes de corregirlo, porque el propio arreglo la destruye):

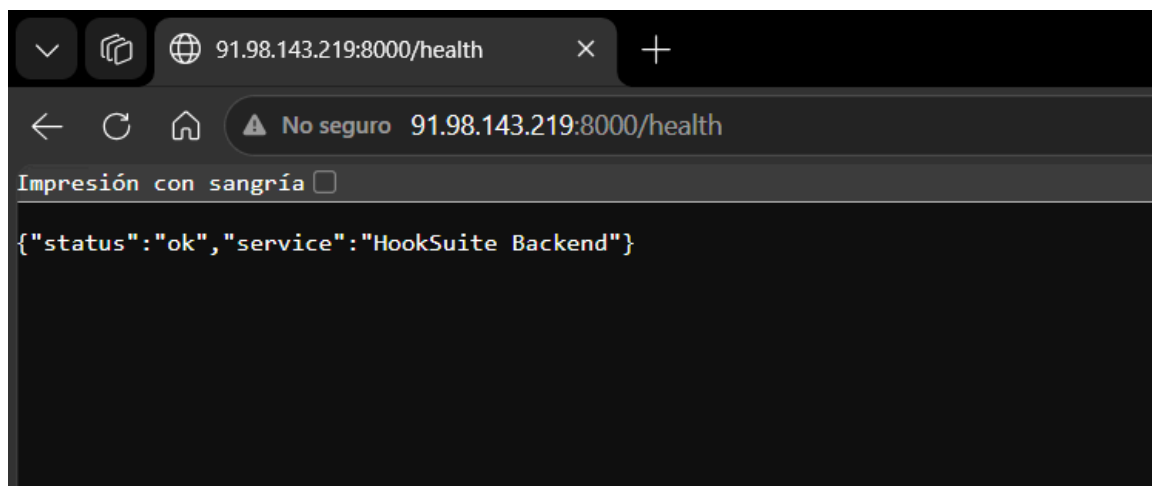


Figura 6: RNF-07 / PR-16 — GET /health de la API responde 200 sin credenciales ni cabecera `www-authenticate`: la API no exige autenticación.

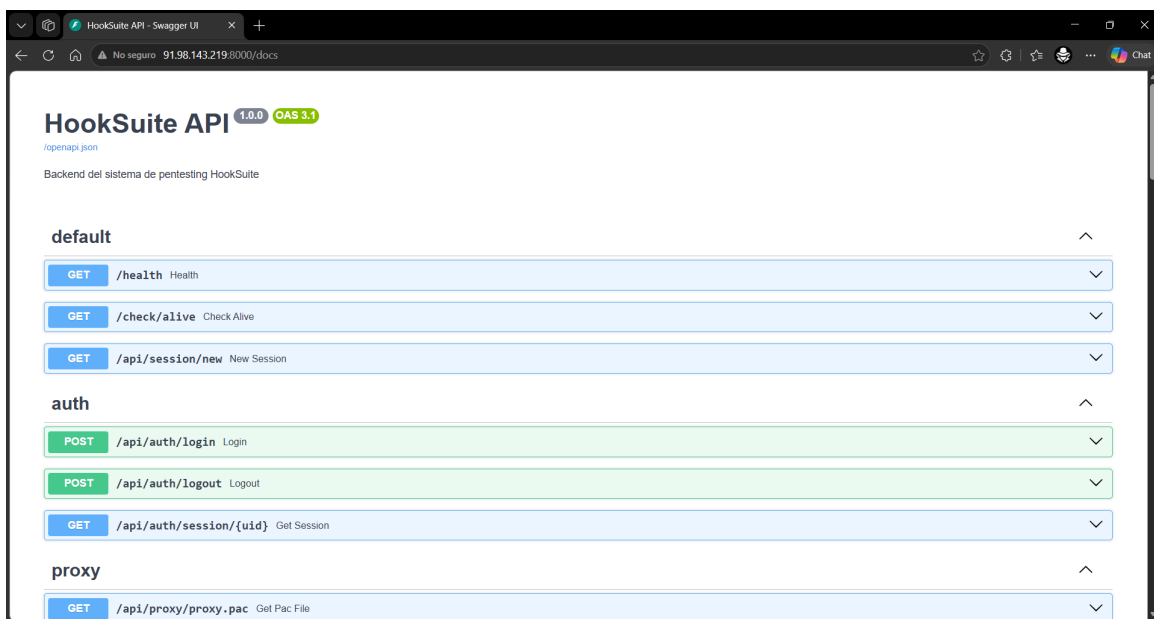


Figura 7: RNF-07 / PR-16 — la documentación Swagger (/docs) queda abierta a cualquiera, exponiendo las 40 operaciones de la API, incluidas las que lanzan tráfico contra terceros.

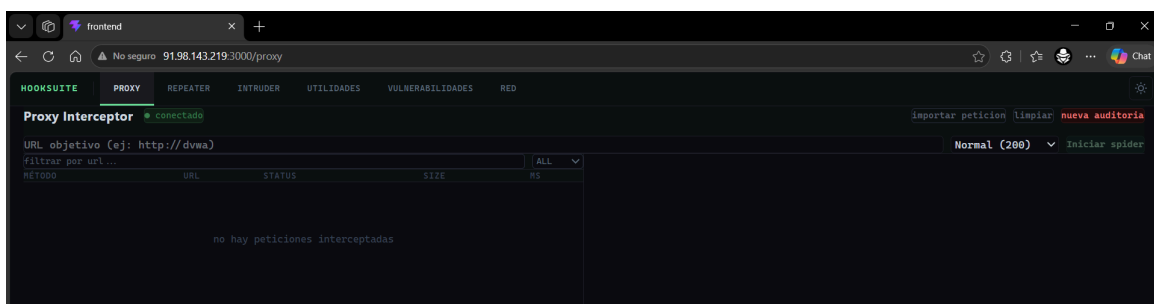


Figura 8: RNF-07 — el frontend servido directamente por :3000 carga 200 sin Basic Auth, evitando por completo el login de Nginx del :80.

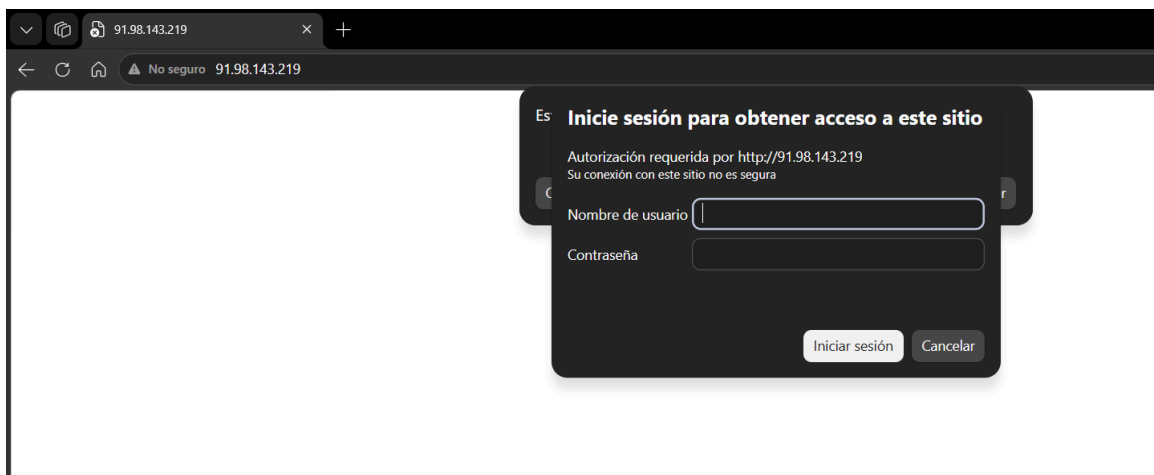


Figura 9: RNF-07 — por contraste, la entrada «correcta» (:80 tras Nginx) sí pide Basic Auth: el control existe, pero se puede rodear por el puerto directo.



Figura 10: RNF-07 — la credencial documentada en el informe de la P1 es rechazada con 401 por nginx/1.31.0: la credencial publicada ya no sirve, lo que refuerza el cambio de modelo de acceso en la Fase 2.

Defectos corregidos: antes y después (RF-03)

Dos pares de capturas tomadas sobre el producto desplegado, antes y después de corregir. El objetivo de las pruebas es una web propia del equipo, autorizada para ello.

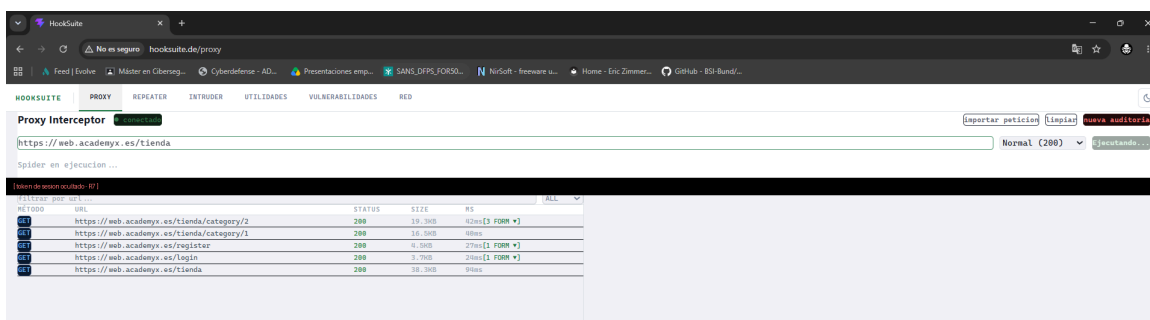


Figura 11: RF-03 — ANTES: el Spider está rastreando («Spider en ejecución...») y el único botón del panel aparece bloqueado como «Ejecutando...». No existe ninguna forma de detener el rastreo desde la interfaz.

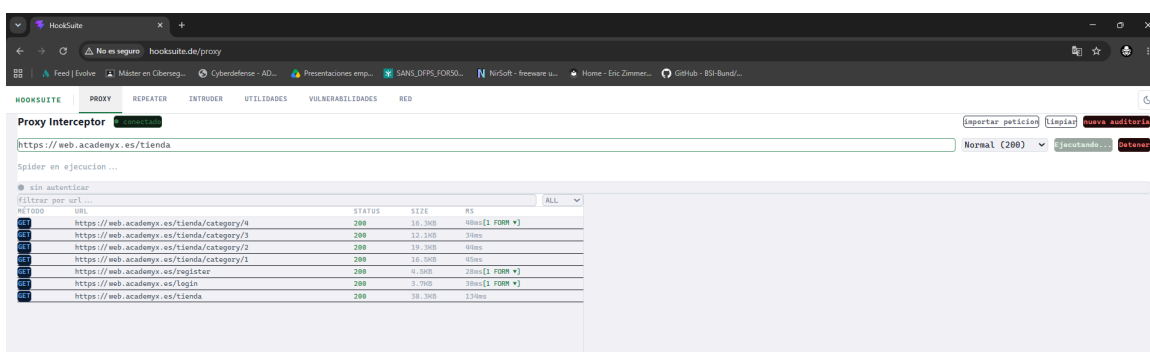


Figura 12: RF-03 — DESPUÉS: en la misma situación aparece el botón «Detener» junto al de ejecución. Obsérvese además el contador «1 FORM» en cada petición, frente a las decenas de la captura siguiente.

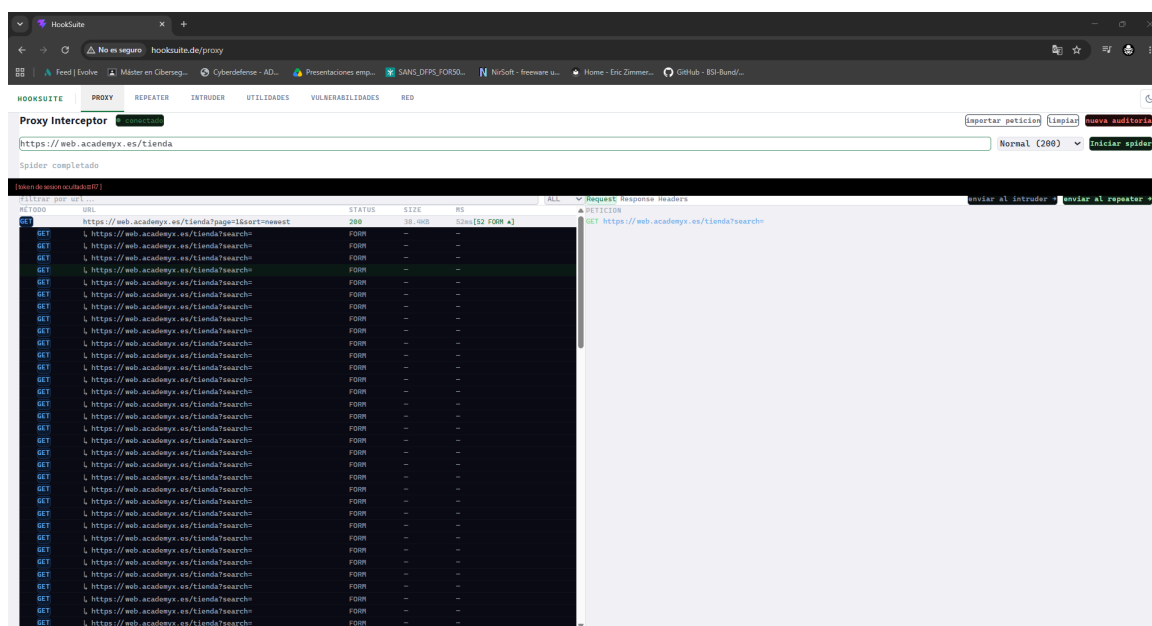


Figura 13: RF-03 — ANTES: una sola petición (/tienda?page=1&sort=newest) arrastra 52 entradas de formulario idénticas, que inundan el panel de resultados.

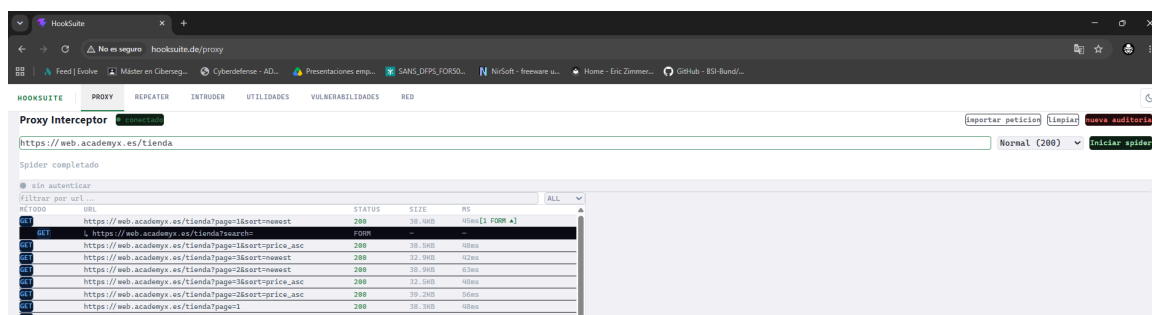


Figura 14: RF-03 — DESPUÉS: la misma petición de la misma página queda en una sola entrada. Solo cambia el contador: 52 → 1.

> **Nota de tratamiento de las capturas (R7).** Las dos del estado anterior mostraban un identificador de sesión de la web de pruebas en la franja superior; se publican con esa franja tapada, y los originales no salen del almacén interno. Las del estado posterior no lo contienen porque la sesión no estaba autenticada.

Condición de carrera detectada antes de reiniciar el servidor

El agente del cortafuegos y el servicio de contenedores no tenían orden de arranque entre sí. Si el segundo ganaba la carrera, **creaba una carpeta donde debía ir el canal de comunicación del agente** —comportamiento normal al montar un fichero que no existe— y el agente quedaba en un bucle de reinicios. El servidor llevaba sin reiniciarse desde mayo, así que nunca se había puesto a prueba. Se corrigió fijando el orden de arranque antes de reiniciar; tras el reinicio, el contador de reinicios del agente marcaba **cero**: la carrera no llegó a producirse.

Otras evidencias ya en mano

- **Incidente de las claves y limpieza del historial:** ensayo de reescritura verificado antes/después (3 → 0 claves), detallado en el apartado 7.
- **Salidas de terminal** (health sin login, openapi.json sin seguridad, cierre de puertos, mensajes de error) guardadas como texto en el repo, no como foto (R7: así ninguna captura publica por error un secreto).

Evidencia de las fugas de aislamiento (RF-12), capturada el 6-oct

Cuatro pruebas lanzadas contra la cocina a través del proxy, como las haría un cliente, **antes de tocar una sola línea de código de la Fase 2**: el propio arreglo borra esta evidencia, y el apartado pide las pruebas que fallaron con su explicación. Los valores de cookie empleados son inventados a propósito, de modo que la captura no contiene ningún identificador real (R7).

Las cuatro confirmaron el fallo: la API responde 200 sin credencial mientras /health responde 401; un token inventado obtiene sesión propia; un segundo usuario lee la cookie de sesión capturada por el primero —y también se obtiene sin token—; y una vulnerabilidad publicada sin token aparece en la sesión de dos auditores distintos.

La captura íntegra, con la causa en fichero y línea de cada una, está en evidencias/fugas-aislamiento-06oct.md. Una corrección respecto a lo que teníamos anotado: la difusión no ocurre solo en las dos llamadas que emiten a todas las sesiones, sino también en los dos endpoints que admiten peticiones sin token, que además **persisten** el dato en la sesión de todos los usuarios. Son cuatro puntos de difusión, no dos.

El aislamiento, verificado después de cerrarlo (RF-12)

Con el sistema de acceso puesto, se repitieron las mismas pruebas contra la cocina. **Once comprobaciones, once en verde**: alta de dos usuarios distintos; entrada de ambos; contraseña incorrecta rechazada con 401 y no con un error del servidor; la identidad del portador del token resuelta correctamente; la API respondiendo con token; 403 al intentar tocar el espacio de otro con token propio —es decir, autenticación y autorización, que son cosas distintas—; el segundo usuario sin ver el hallazgo del primero; y el segundo usuario sin ver la cookie de sesión del primero, que era la tercera fuga.

Se probó además **como ataque**, no solo como comprobación: un usuario autenticado intentó dirigir el Spider al espacio de otro nombrándolo en el cuerpo de la petición, que era el quinto fallo del apartado 7. El espacio de la víctima quedó vacío y el rastreo fue a parar al del atacante.

Ni el código de invitación ni las contraseñas aparecen en ninguna salida: el guion de pruebas los mantiene en variables (R7).

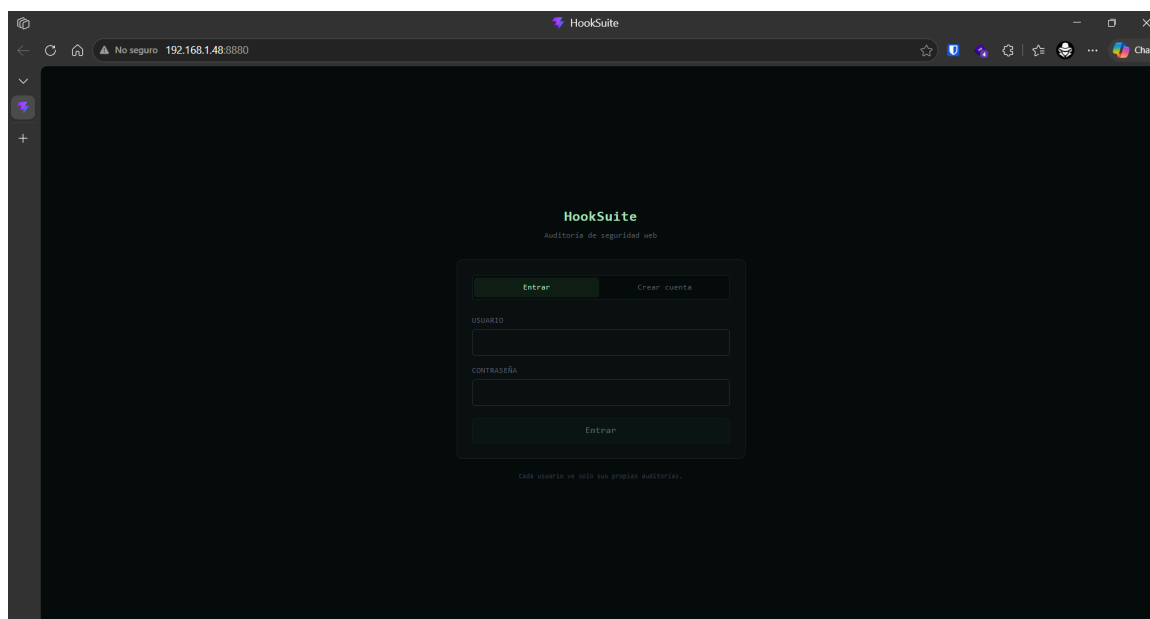


Figura 15: RF-12 — la entrada del producto tras retirar el Basic Auth de Nginx: un panel de acceso propio, con pestañas «Entrar» y «Crear cuenta». Antes, la raíz devolvía 401 pidiendo una credencial que ningún integrante del grupo conocía.

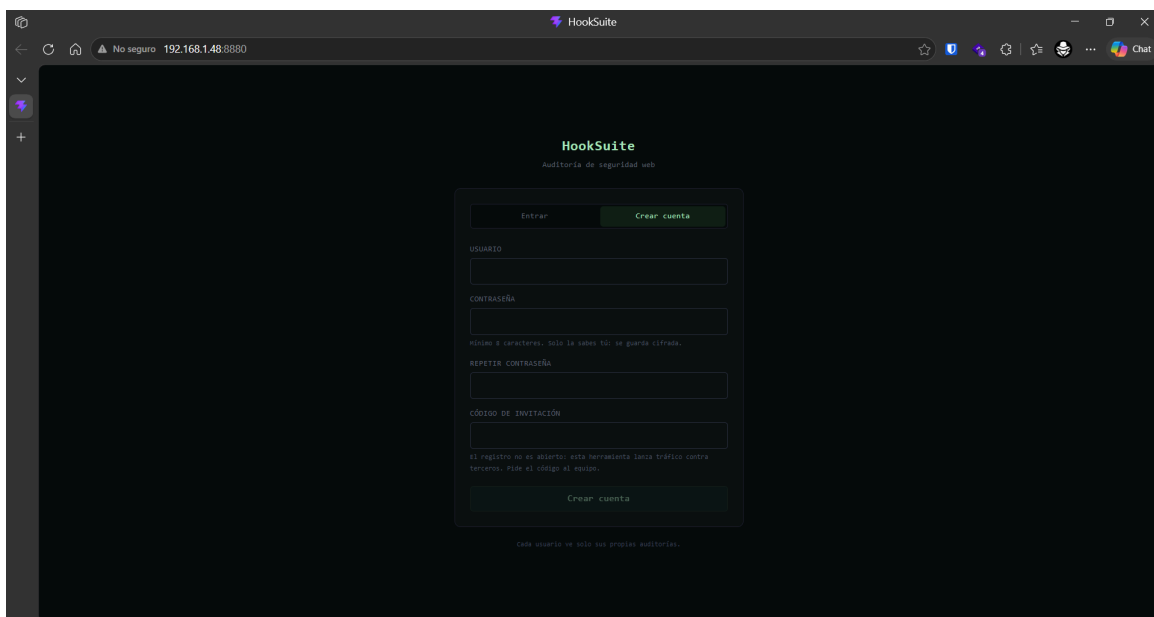


Figura 16: RF-12 — el alta exige **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, así que con altas anónimas cualquiera podría atacar desde la infraestructura del grupo. La contraseña la elige el usuario y solo la conoce él.

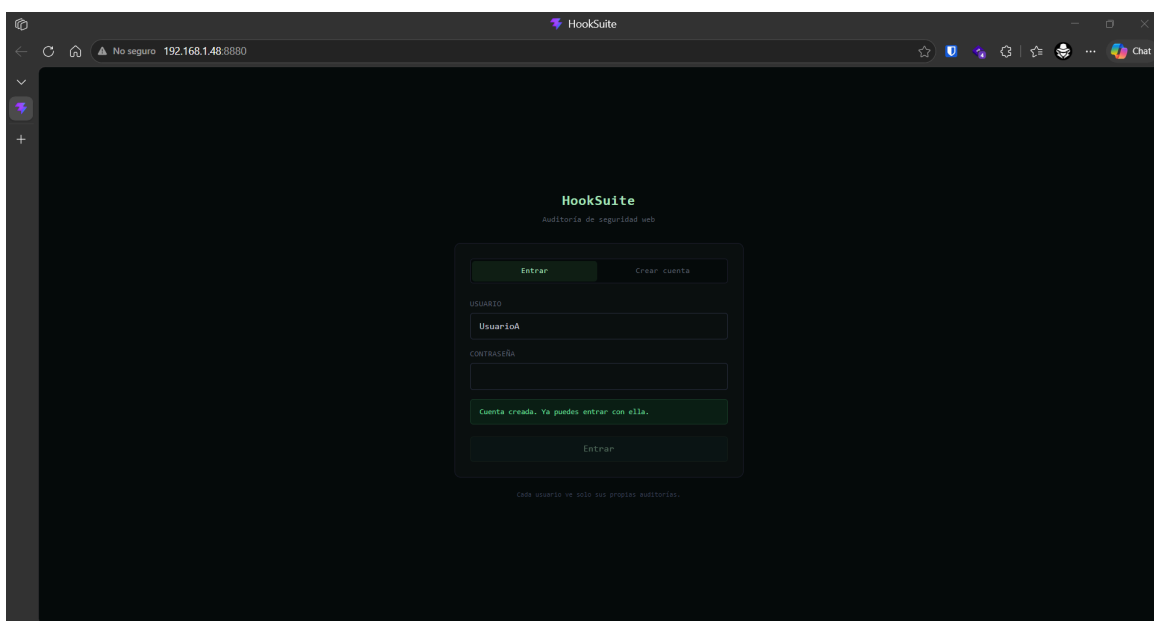


Figura 17: RF-12 — alta completada: la interfaz confirma la creación de la cuenta y devuelve al formulario de entrada. Cubre la mitad «registro» del requisito, que el enunciado pide junto al login.

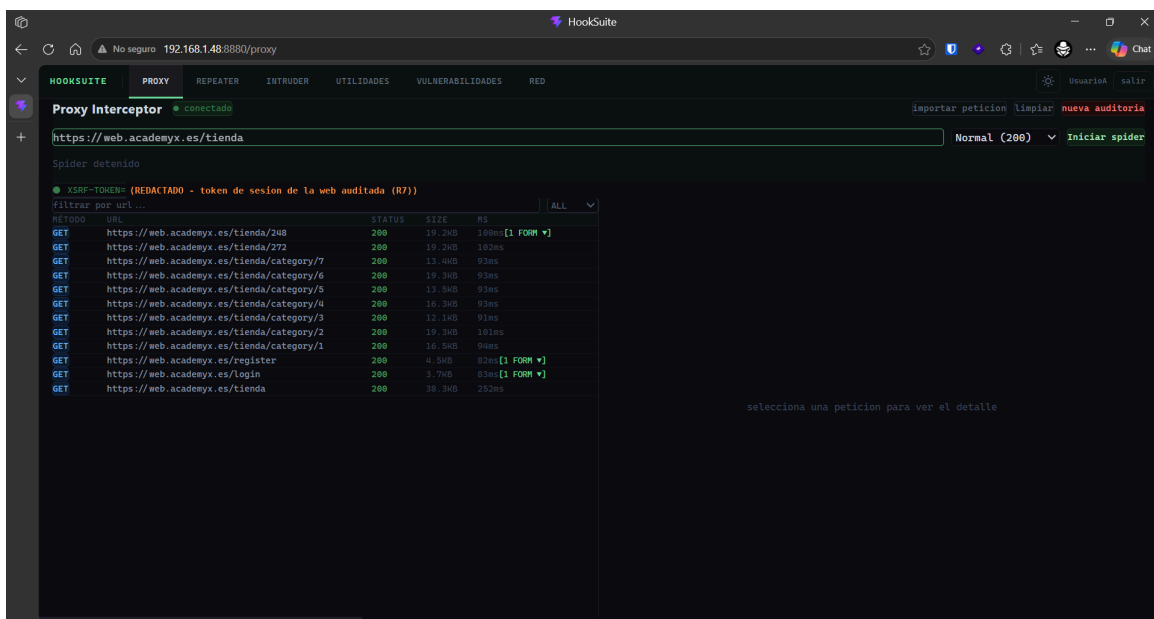


Figura 18: RF-12 — ANTES de la Fase 2 este panel era común a todos. Ahora el usuario A, autenticado, ve las trece peticiones que su propio Spider capturó del objetivo autorizado. Compárese con la captura siguiente, tomada al mismo tiempo.

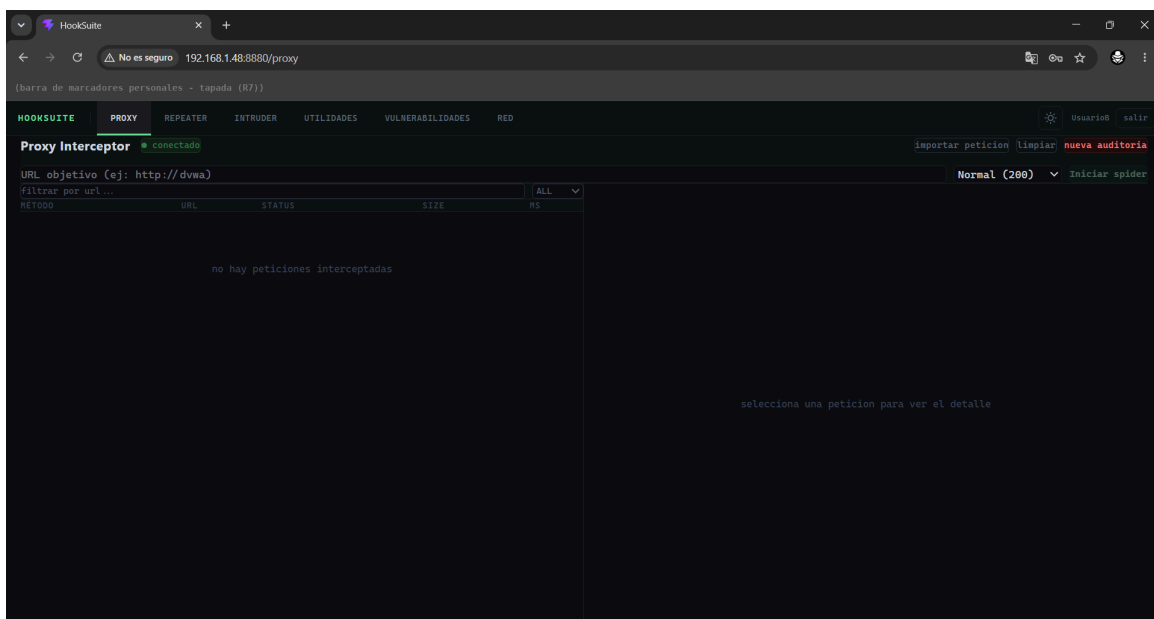


Figura 19: RF-12 — DESPUÉS, el contraste: el usuario B, autenticado a la vez y en otro navegador, ve su panel vacío. Antes de la Fase 2 habría visto exactamente lo mismo que el usuario A. Este par es la prueba del aislamiento por usuario.

> **Nota de tratamiento de las capturas (R7).** La del usuario A mostraba el valor íntegro de la cabecera XSRF-TOKEN, que es el identificador de sesión capturado **de la web auditada**: justo el dato que este requisito existe para no compartir. Se publica con ese valor tapado y la etiqueta a la vista, para que se vea qué se redactó; el original no sale del almacén interno. La del usuario B lleva tapada la barra de marcadores del navegador, ajena al proyecto.

El despliegue habría terminado «en verde» sin aplicar la configuración del proxy

La comprobación previa al despliegue de la Fase 2 —hecha **antes** de tocar la instalación pública— encontró un defecto no en el producto, sino **en el propio plan de despliegue**, escrito dos días antes. Se cuenta porque el enunciado pide las pruebas que fallaron, y porque el modo de fallo es de los que no avisan.

El plan mandaba reconstruir «solo el frontend, el backend y el proxy». Pero el proxy **no se construye**: usa una imagen descargada, y su bloque del fichero de composición **no cambiaba ni un byte** entre la

rama publicada y la que se iba a desplegar. Compose lo veía idéntico a lo que ya estaba corriendo y, por tanto, **no lo habría recreado**. Y como el proxy lee su configuración una sola vez al arrancar —y el contenedor en producción llevaba vivo desde dos días antes—, el despliegue habría acabado **sin errores y sin aplicar nada**: el panel de acceso nuevo habría seguido detrás de la contraseña compartida que la Fase 2 venía a jubilar, y las dos rutas retiradas habrían seguido publicadas.

Lo que convierte esto en una lección y no en una anécdota: era el mismo fallo que ya había ocurrido dos días antes, con el mismo servicio y la misma causa. Y la razón de que se repitiera es instructiva: **nombrar un servicio en un comando da la impresión de cubrirlo**. El plan lo nombraba, así que nadie volvió a mirar si el comando lo tocaba de verdad.

Y el 10 de octubre apareció el caso espejo, que completa la lección. Al revés que entonces, se recrearon el frontend, la API y el modulo de IA **y no el proxy** —porque su configuración no había cambiado—. Resultado: **502 en todo**. Los contenedores recreados reciben **direcciones nuevas**, y el proxy resuelve los nombres de sus destinos **una sola vez, al arrancar**: seguía llamando a las direcciones de los contenedores que acababan de desaparecer. Su registro lo decía con todas las letras, nombrando direcciones que ya no existían.

Las dos mitades tienen la misma raíz: **el estado del proxy se fija al arrancar**, tanto la configuración que lee como las direcciones que resuelve. De ahí sale una regla operativa que no estaba escrita en ningún sitio y ahora sí: **cuando se recrea cualquiera de los servicios a los que el proxy apunta, se recrea también el proxy**. No es una precaución: es que no hacerlo lo deja hablando con direcciones muertas.

Arreglo: forzar explícitamente la recreación de ese servicio. Cuesta segundos y no construye nada. **Se descartó** recargar la configuración en caliente —que también habría funcionado, pero depende de un detalle del sistema de ficheros en lugar de ser incondicional— y también tocar su bloque del fichero de composición solo para que Compose lo viera cambiado, que es resolver el síntoma ensuciando la fuente.

Un modulo que parecia sano y no podia hablar con nadie (RF-08)

La prueba mas barata de esta practica fue una peticion con curl, y descubrio un defecto que llevaba cuatro dias en produccion sin que nadie lo notara: **el modulo de IA no podia comunicarse con el backend desde la Fase 2**.

Ese modulo preguntaba al backend cada cinco segundos si habia trabajo. Al poner el control de acceso, esa ruta quedo protegida junto con las demas —correctamente—, y el modulo no presenta credencial ninguna, de modo que recibia un rechazo. Hay ademas un segundo muro detras del primero: incluso con una credencial valida, el control comprueba que el espacio de datos nombrado en la ruta sea el tuyo, y el modulo usaba un identificador fijo que no es de ningún usuario.

Lo grave no es el defecto, es por que no se vio. El código comprobaba si la respuesta era un éxito; un rechazo por credencial **no provoca ningún error**, simplemente no es un éxito, así que el bucle volvía a dormir cinco segundos y lo intentaba otra vez. Y la comprobación de arranque —que esperaba al backend hasta treinta veces— **descartaba su propio resultado** y continuaba igual, escribiendo en el registro que estaba esperando instrucciones. Un operador que mirara ese registro habría visto un modulo en marcha y a la espera. Estaba en marcha, y no podía recibir nada.

Es el mismo patrón que RNF-06 vino a cerrar dentro del clasificador, un nivel mas arriba: **un fallo que no se distingue de la normalidad**. Que apareciera dos veces en el mismo modulo, por caminos distintos, sugiere que no es un descuido aislado sino una forma de escribir —tratar el camino de error como un caso que «ya se vera»— y es la razón por la que ahora el modulo falla ruidosamente: comprueba el bus al arrancar y se detiene si no esta.

Como se midio, y por que importa el metodo. Tres peticiones sin credencial contra la instalación de pruebas: la consulta de trabajo pendiente, la comprobación de arranque y el envío de una orden. Las tres rechazadas. La evidencia se guardo **como texto** y no como captura, de modo que es comparable con la de después. Y el alcance del problema —que el camino de vuelta de los resultados estaba igual de roto— se estableció **sin abrir el fichero del orquestador**, que esta en la lista de material que este equipo no vuelca a la conversación con la herramienta de IA (apartado 12): basto contar sus referencias al backend y mirar su esqueleto de funciones.

Evidencia completa en docs/evidencias/canal-ia-backend-muerto-10oct.md.

Las dos contrapartidas del bus, probadas antes de confiar en el

Reconectar el modulo por un bus interno (apartado 5) cambia la forma del riesgo: un control de acceso falla cerrando, un bus falla **abriendo**. Así que lo que se prueba no es solo que el reparto funcione, sino que **no se puede repartir mal**. Dieciocho casos en dos conjuntos, con dobles para el bus, el gestor de sesiones y el orquestador: **ni red, ni credencial de modelo, ni contenedores, cero coste**.

Los dos que justifican el esfuerzo:

- **Un hallazgo sin dueño se descarta**, no se guarda ni se emite. Sin esa regla, un mensaje mal formado reabriría exactamente la fuga que cerro la Fase 2 —un hallazgo publicado sin credencial apareciendo en el panel de dos auditores distintos—.
- **Dos auditores no ven lo del otro**, comprobado repartiendo a dos espacios y verificando que cada uno recibe solo lo suyo.

Y tres mas que cubren el fallo silencioso en el otro extremo: si la auditoria se interrumpe, **se publica el motivo** en lugar de no publicar nada; si nadie escucha el bus, publicar **lo dice** en vez de dar el trabajo por encolado; y si una orden llega sin dueño, **no se audita**, porque no habría a quien devolverle el resultado.

Reproducible con `python3 backend/tests/test_bus_reparto.py` y `python3 ia/tests/test_bus_publicacion.py`.

El recorrido completo del disparador, verificado de punta a punta (RF-08)

Durante cuatro días el disparador estuvo **construido y sin poder demostrarse**: el botón existía, el bus estaba cableado, y aun así una pulsación desde el panel no movía ni una línea del módulo de IA. El apartado llegó a listar tres hipótesis sin poder decidir entre ellas.

El 10-oct se cerró, y la forma de cerrarlo es parte del resultado: en lugar de seguir estimando sobre ventanas de tiempo aproximadas, **se anotó la hora exacta de la pulsación** y se midió contra ella. A partir de las 13:58:10 el servidor registra **una** petición al disparador, con respuesta correcta, y el registro del módulo de IA pasa de 192 a 376 líneas. Panel → autenticación → bus → módulo suscrito: el recorrido entero, por primera vez.

Lo que no se da por explicado. Cincuenta y ocho minutos antes, la misma pulsación no había movido nada, y entre una y otra **no se tocó el producto** —ningún fichero ni contenedor cambió—. La explicación cómoda sería una sesión caducada, pero no encaja: el sondeo de instrucciones sigue devolviendo **no autorizado** ahora, con sesión válida y el disparador funcionando. Queda abierto y declarado, porque un apartado de pruebas que explica lo que no sabe explicar vale menos que uno que lo admite.

El techo de gasto, ejecutado: una auditoría entera por 0 € (RF-08)

Este caso estuvo declarado durante semanas como **preparado y no ejecutado**, y se deja constancia de que ya no lo está. Una auditoría del módulo de IA no tiene límite de consultas al modelo: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Con el disparador funcionando, cada pulsación arranca ese recorrido completo.

El caso estaba diseñado para costar **cero**: fijando el techo en 0 por variable de entorno, ninguna consulta debe salir y la auditoría debe quedar marcada como trunca. **Ejecutado el 10-oct desde el panel, con sesión real**. La auditoría recorrió cuarenta y ocho elementos, no salió **ni una sola** consulta al modelo —coste 0 €— y los cuarenta y ocho quedaron marcados como no analizados, cada uno con el motivo del techo. El techo no se limita a cortar: **el corte se ve**, que era la condición que se le puso al diseñarlo.

La mitad que falta del caso, y se declara: el techo en 1, donde debe pasar una sola consulta y el resto quedar como «no analizado». Esa sí necesita clave de API válida, porque la consulta que pasa tiene que salir de verdad.

 **HUECO** — responsable: **José María + Claude**. Ejecutar la segunda mitad del caso del techo —techo en 1— en cuanto haya clave de API válida: debe pasar una sola consulta y el resto quedar

como «no analizado» con el motivo del techo. La primera mitad (techo 0) ya está ejecutada y capturada. Coste previsto: una consulta.

AISLAMIENTO POR USUARIO VERIFICADO EN LA COCINA (11/11) · PENDIENTE DE VERIFICAR EN PRODUCCIÓN

EVIDENCIA DE EXPOSICIÓN Y DE LA FASE 1 COMPLETA

⚠ **HUECO** — responsable: **José María + Claude**. Plan de pruebas formal y capturas de PRODUCTO de los bugs (500 de `check/alive`, trampa del Spider) reproducidos en la cocina. Las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. Las pruebas las hacen José María y Claude (R2).

Operación degradada (RNF-06), probada sin gastar saldo

La prueba que demuestra RNF-06 no necesita ni clave de API ni conexión: sustituye el cliente del modelo por uno simulado y comprueba qué devuelve el clasificador en cada situación. Ocho casos: los tres estados sobre la vía del proxy, el estado degradado en las cuatro vías, el camino correcto de la identificación de servidor, y que se le entrega un esquema de respuesta al cliente.

El valor de la prueba está en haberla visto fallar. Ejecutada contra el código anterior devuelve 0 de 8, y el motivo es revelador: los cuatro métodos llamaban a `.get()` sobre el objeto que el cliente reescrito devuelve ahora, de modo que la fase de análisis no es que fallara en silencio, es que se interrumpía entera. Contra el código corregido devuelve 8 de 8. Una prueba que pasa siempre no prueba nada; esta distingue las dos versiones.

Por qué una prueba y no una revisión visual. El fichero del clasificador está en la lista de material que este equipo no vuelca a la conversación con la herramienta de IA (apartado 12), así que se editó sin leerlo, mediante una receta de pasos verificables. En ese modo de trabajo la prueba no es un complemento de la revisión: es lo que la sustituye, y es más estricta, porque comprueba el comportamiento en vez de la impresión que deja el código al leerlo.

Reproducible con `python3 ia/tests/test_tres_estados.py`. Evidencia completa, con las dos salidas, en `docs/evidencias/rnf06-tres-estados-antes-y-despues-09oct.md`.

RNF-06 visto en el panel, que es donde tenía que verse

Una prueba unitaria demuestra que el clasificador **devuelve** el estado degradado; no demuestra que alguien lo **enseñe**. Entre las dos cosas hubo un hueco real: durante un día el estado existió en el módulo de IA y no lo consumía nadie al otro lado, de modo que RNF-06 estaba cumplido y era invisible. El 10-oct se cerró el hueco y se comprobó en la interfaz, con una auditoría lanzada desde el panel y sesión real.

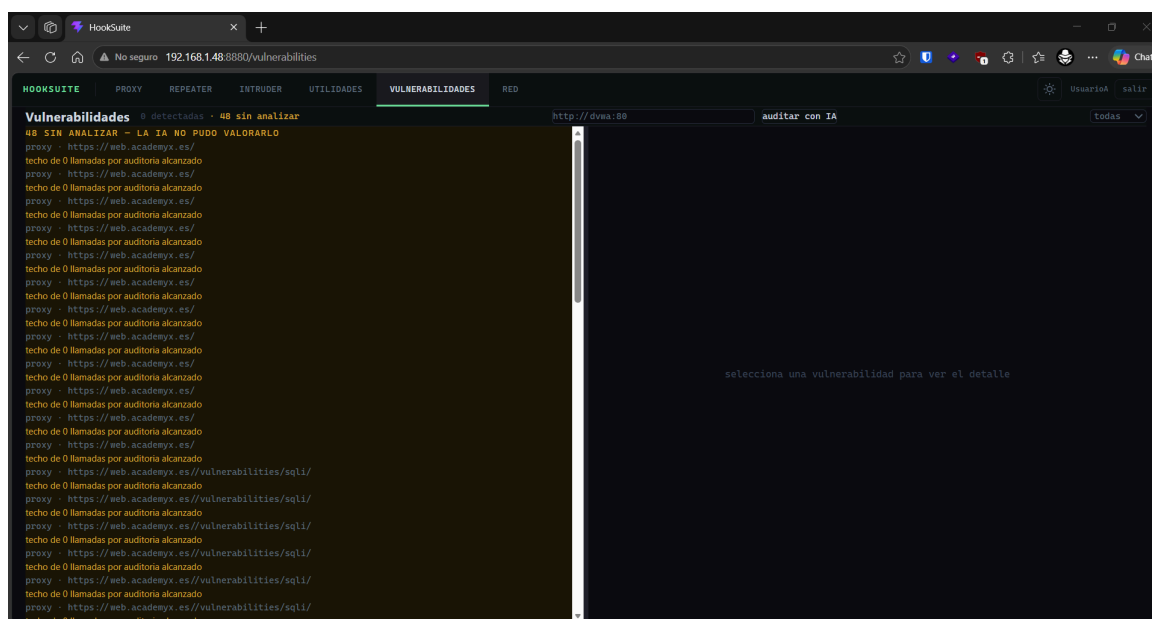


Figura 20: RNF-06 / RF-08 — el panel tras una auditoría truncada por el techo de gasto. El encabezado anuncia **48 sin analizar — la IA no pudo valorarlo** y cada entrada lleva su motivo, *techo de 0 llamadas por auditoria alcanzado*: el recorte se ve en lugar de pasar por «sin hallazgos». El recuento *0 detectadas · 48 sin analizar* distingue las dos cosas. Los cuarenta y ocho elementos provienen de un rastreo previo sobre **web.academyx.es**, web propia del equipo levantada para practicar; el objetivo que figura en la barra, *http://dwva:80*, es el de la auditoría siguiente.

Lo que la figura prueba, y que ninguna prueba unitaria alcanzaba: que el mensaje de lista vacía **dejó de mentir**. Antes, una auditoría que no había podido analizar nada y otra que había analizado y no encontrado nada producían la misma pantalla. Ahora el aviso va aparte de los hallazgos —en su propia tira, por encima— porque mezclarlo obligaría a inventarle una severidad a algo que precisamente **no se pudo valorar**.

Lo que el modelo puede devolver, y lo que el código daba por hecho (RF-08)

Las cuatro vías de análisis del módulo de IA piden al modelo una respuesta con un formato fijo, y ese formato se le impone mediante un **esquema** que viaja con la consulta: la API garantiza que la respuesta se ajusta a él. El esquema, entonces, no es documentación: **es el control**. Y un campo declarado con tipo abierto no es un campo permisivo, es un campo sin control.

La prueba comprueba que el esquema obliga a las dos cosas que las instrucciones piden: que la bandera de resultado sea un booleano y que la confianza venga en escala de 0 a 100. Contra el código anterior devuelve *5 de 11*, y los fallos no eran cosméticos:

1. De los tres campos de bandera —uno por vía, con **tres nombres distintos para el mismo papel**—, solo el primero estaba declarado como booleano. Los otros dos admitían cadena de texto, y el clasificador evalúa los tres igual, dentro de un `if`. En el lenguaje en que está escrito, **cualquier cadena no vacía es verdadera, incluida la palabra que significa «no»**: una respuesta negativa del modelo se habría registrado como vulnerabilidad confirmada.
2. La confianza se declaraba como número **sin rango**, así que nada rechazaba un valor fuera de la escala.
3. El esquema de la vía de identificación de servidor declaraba **cero campos**, mientras sus instrucciones piden nueve, uno de ellos una lista de objetos.

La causa era una sola, y estaba fuera del repositorio del producto. Los esquemas no se escriben a mano: los genera una herramienta de la práctica que deduce los campos a partir de las lecturas que el propio clasificador hace sobre la respuesta. Esa herramienta llevaba una lista, escrita a mano, de los campos «que el código interpreta», y se redactó mirando **una sola** de las cuatro vías; su propio comentario lo delata, porque habla de la bandera en singular. El esquema vacío tiene la misma raíz por el camino opuesto: la vía de identificación de servidor devuelve la respuesta del modelo **sin**

inspeccionarla, de modo que no hay ninguna lectura de la que deducir nada. Un esquema no se puede derivar de un código que no mira los datos.

El arreglo fue por tanto en el generador —no en el fichero generado, que lleva en su cabecera la advertencia de que se regenera— y después se regeneró. La prueba pasa *11 de 11*, y la de RNF-06 sigue en *8 de 8*: el cambio no rompió nada.

Reproducible con `python3 ia/tests/test_esquemas_bandera.py ia/esquemas.py`. Evidencia con las dos salidas, antes y después, en `docs/evidencias/esquemas-bandera-antes-10oct.md`.

Un fallo silencioso no es un fallo menor: la escala de la confianza (RF-08 / RNF-06)

El filtro por confianza compara el valor que devuelve el modelo contra un umbral. Las instrucciones piden ese valor en escala de 0 a 100, pero **ninguna de las cuatro pide que sea un número entero** —se comprobó leyendo las cuatro—, y ahí había una rendija que el esquema **no puede** cerrar: si el modelo devolviera la escala de 0 a 1, un valor de 0,85 queriendo decir «85 %» está **dentro** del rango permitido, de modo que pasaría el control y después quedaría por debajo del umbral. El hallazgo **desaparecería sin dejar rastro**, que es exactamente la situación que RNF-06 existe para evitar. El caso peor es el valor 1: en escala de 0 a 1 significa certeza total, y leído como «1 %» descartaría un hallazgo seguro.

Dónde se arregló, y por qué no en el esquema. Imponer «entero» en el esquema habría exigido algo que las instrucciones no piden, y habría rechazado un valor decimal legítimo dentro de 0 a 100. El arreglo va en el clasificador: la franja entre 0 y 1 se trata como **sospecha de escala** y se encamina al estado degradado de RNF-06, con su motivo. Así el problema **se ve** en el informe en lugar de corregirse a la fuerza o desaparecer.

La prueba distingue las dos versiones, que es la única forma de que valga: contra el código corregido devuelve *19 de 19*; contra el anterior, *6 de 19*. Y los seis que pasan en la versión antigua son precisamente los casos legítimos, lo que demuestra que la guarda no introduce falsos positivos. El caso que lo prueba todo es el que comprueba que un hallazgo con confianza 0,85 **no** se devuelve como «sin vulnerabilidad»: falla contra el código anterior, es decir, el descarte silencioso queda **demostrado** y no argumentado.

Reproducible con `python3 ia/tests/test_escala_confianza.py`. Evidencia con las dos salidas en `docs/evidencias/escala-confianza-guardia-10oct.md`.

9. Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código (cocina local, 4-oct). La columna de evidencia enlaza la figura de la memoria y el **minuto exacto del vídeo**; se rellena al grabar.

Req.	Descripción	Implementación (ruta)	Evidencia
RF-01	Dashboard web sin instalación	PR-01 PR-02 PR-03 PR-04 PR-05 PR-06 PR-07 PR-08 PR-09 PR-10 PR-11 PR-12 PR-13 PR-14 PR-15 PR-16 PR-17 PR-18 PR-19 PR-20 PR-21 PR-22 PR-23 PR-24 PR-25 PR-26 PR-27 PR-28 PR-29 PR-30 PR-31 PR-32 PR-33 PR-34 PR-35 PR-36 PR-37 PR-38 PR-39 PR-40 PR-41 PR-42 PR-43 PR-44 PR-45 PR-46 PR-47 PR-48 PR-49 PR-50 PR-51 PR-52 PR-53 PR-54 PR-55 PR-56 PR-57 PR-58 PR-59 PR-60 PR-61 PR-62 PR-63 PR-64 PR-65 PR-66 PR-67 PR-68 PR-69 PR-70 PR-71 PR-72 PR-73 PR-74 PR-75 PR-76 PR-77 PR-78 PR-79 PR-80 PR-81 PR-82 PR-83 PR-84 PR-85 PR-86 PR-87 PR-88 PR-89 PR-90 PR-91 PR-92 PR-93 PR-94 PR-95 PR-96 PR-97 PR-98 PR-99 PR-100 PR-101 PR-102 PR-103 PR-104 PR-105 PR-106 PR-107 PR-108 PR-109 PR-110 PR-111 PR-112 PR-113 PR-114 PR-115 PR-116 PR-117 PR-118 PR-119 PR-120 PR-121 PR-122 PR-123 PR-124 PR-125 PR-126 PR-127 PR-128 PR-129 PR-130 PR-131 PR-132 PR-133 PR-134 PR-135 PR-136 PR-137 PR-138 PR-139 PR-140 PR-141 PR-142 PR-143 PR-144 PR-145 PR-146 PR-147 PR-148 PR-149 PR-150 PR-151 PR-152 PR-153 PR-154 PR-155 PR-156 PR-157 PR-158 PR-159 PR-160 PR-161 PR-162 PR-163 PR-164 PR-165 PR-166 PR-167 PR-168 PR-169 PR-170 PR-171 PR-172 PR-173 PR-174 PR-175 PR-176 PR-177 PR-178 PR-179 PR-180 PR-181 PR-182 PR-183 PR-184 PR-185 PR-186 PR-187 PR-188 PR-189 PR-190 PR-191 PR-192 PR-193 PR-194 PR-195 PR-196 PR-197 PR-198 PR-199 PR-200 PR-201 PR-202 PR-203 PR-204 PR-205 PR-206 PR-207 PR-208 PR-209 PR-210 PR-211 PR-212 PR-213 PR-214 PR-215 PR-216 PR-217 PR-218 PR-219 PR-220 PR-221 PR-222 PR-223 PR-224 PR-225 PR-226 PR-227 PR-228 PR-229 PR-230 PR-231 PR-232 PR-233 PR-234 PR-235 PR-236 PR-237 PR-238 PR-239 PR-240 PR-241 PR-242 PR-243 PR-244 PR-245 PR-246 PR-247 PR-248 PR-249 PR-250 PR-251 PR-252 PR-253 PR-254 PR-255 PR-256 PR-257 PR-258 PR-259 PR-260 PR-261 PR-262 PR-263 PR-264 PR-265 PR-266 PR-267 PR-268 PR-269 PR-270 PR-271 PR-272 PR-273 PR-274 PR-275 PR-276 PR-277 PR-278 PR-279 PR-280 PR-281 PR-282 PR-283 PR-284 PR-285 PR-286 PR-287 PR-288 PR-289 PR-290 PR-291 PR-292 PR-293 PR-294 PR-295 PR-296 PR-297 PR-298 PR-299 PR-300 PR-301 PR-302 PR-303 PR-304 PR-305 PR-306 PR-307 PR-308 PR-309 PR-310 PR-311 PR-312 PR-313 PR-314 PR-315 PR-316 PR-317 PR-318 PR-319 PR-320 PR-321 PR-322 PR-323 PR-324 PR-325 PR-326 PR-327 PR-328 PR-329 PR-330 PR-331 PR-332 PR-333 PR-334 PR-335 PR-336 PR-337 PR-338 PR-339 PR-340 PR-341 PR-342 PR-343 PR-344 PR-345 PR-346 PR-347 PR-348 PR-349 PR-350 PR-351 PR-352 PR-353 PR-354 PR-355 PR-356 PR-357 PR-358 PR-359 PR-360 PR-361 PR-362 PR-363 PR-364 PR-365 PR-366 PR-367 PR-368 PR-369 PR-370 PR-371 PR-372 PR-373 PR-374 PR-375 PR-376 PR-377 PR-378 PR-379 PR-380 PR-381 PR-382 PR-383 PR-384 PR-385 PR-386 PR-387 PR-388 PR-389 PR-390 PR-391 PR-392 PR-393 PR-394 PR-395 PR-396 PR-397 PR-398 PR-399 PR-400 PR-401 PR-402 PR-403 PR-404 PR-405 PR-406 PR-407 PR-408 PR-409 PR-410 PR-411 PR-412 PR-413 PR-414 PR-415 PR-416 PR-417 PR-418 PR-419 PR-420 PR-421 PR-422 PR-423 PR-424 PR-425 PR-426 PR-427 PR-428 PR-429 PR-430 PR-431 PR-432 PR-433 PR-434 PR-435 PR-436 PR-437 PR-438 PR-439 PR-440 PR-441 PR-442 PR-443 PR-444 PR-445 PR-446 PR-447 PR-448 PR-449 PR-450 PR-451 PR-452 PR-453 PR-454 PR-455 PR-456 PR-457 PR-458 PR-459 PR-460 PR-461 PR-462 PR-463 PR-464 PR-465 PR-466 PR-467 PR-468 PR-469 PR-470 PR-471 PR-472 PR-473 PR-474 PR-475 PR-476 PR-477 PR-478 PR-479 PR-480 PR-481 PR-482 PR-483 PR-484 PR-485 PR-486 PR-487 PR-488 PR-489 PR-490 PR-491 PR-492 PR-493 PR-494 PR-495 PR-496 PR-497 PR-498 PR-499 PR-500 PR-501 PR-502 PR-503 PR-504 PR-505 PR-506 PR-507 PR-508 PR-509 PR-510 PR-511 PR-512 PR-513 PR-514 PR-515 PR-516 PR-517 PR-518 PR-519 PR-520 PR-521 PR-522 PR-523 PR-524 PR-525 PR-526 PR-527 PR-528 PR-529 PR-530 PR-531 PR-532 PR-533 PR-534 PR-535 PR-536 PR-537 PR-538 PR-539 PR-540 PR-541 PR-542 PR-543 PR-544 PR-545 PR-546 PR-547 PR-548 PR-549 PR-550 PR-551 PR-552 PR-553 PR-554 PR-555 PR-556 PR-557 PR-558 PR-559 PR-560 PR-561 PR-562 PR-563 PR-564 PR-565 PR-566 PR-567 PR-568 PR-569 PR-570 PR-571 PR-572 PR-573 PR-574 PR-575 PR-576 PR-577 PR-578 PR-579 PR-580 PR-581 PR-582 PR-583 PR-584 PR-585 PR-586 PR-587 PR-588 PR-589 PR-590 PR-591 PR-592 PR-593 PR-594 PR-595 PR-596 PR-597 PR-598 PR-599 PR-600 PR-601 PR-602 PR-603 PR-604 PR-605 PR-606 PR-607 PR-608 PR-609 PR-610 PR-611 PR-612 PR-613 PR-614 PR-615 PR-616 PR-617 PR-618 PR-619 PR-620 PR-621 PR-622 PR-623 PR-624 PR-625 PR-626 PR-627 PR-628 PR-629 PR-630 PR-631 PR-632 PR-633 PR-634 PR-635 PR-636 PR-637 PR-638 PR-639 PR-640 PR-641 PR-642 PR-643 PR-644 PR-645 PR-646 PR-647 PR-648 PR-649 PR-650 PR-651 PR-652 PR-653 PR-654 PR-655 PR-656 PR-657 PR-658 PR-659 PR-660 PR-661 PR-662 PR-663 PR-664 PR-665 PR-666 PR-667 PR-668 PR-669 PR-670 PR-671 PR-672 PR-673 PR-674 PR-675 PR-676 PR-677 PR-678 PR-679 PR-680 PR-681 PR-682 PR-683 PR-684 PR-685 PR-686 PR-687 PR-688 PR-689 PR-690 PR-691 PR-692 PR-693 PR-694 PR-695 PR-696 PR-697 PR-698 PR-699 PR-700 PR-701 PR-702 PR-703 PR-704 PR-705 PR-706 PR-707 PR-708 PR-709 PR-710 PR-711 PR-712 PR-713 PR-714 PR-715 PR-716 PR-717 PR-718 PR-719 PR-720 PR-721 PR-722 PR-723 PR-724 PR-725 PR-726 PR-727 PR-728 PR-729 PR-730 PR-731 PR-732 PR-733 PR-734 PR-735 PR-736 PR-737 PR-738 PR-739 PR-740 PR-741 PR-742 PR-743 PR-744 PR-745 PR-746 PR-747 PR-748 PR-749 PR-750 PR-751 PR-752 PR-753 PR-754 PR-755 PR-756 PR-757 PR-758 PR-759 PR-760 PR-761 PR-762 PR-763 PR-764 PR-765 PR-766 PR-767 PR-768 PR-769 PR-770 PR-771 PR-772 PR-773 PR-774 PR-775 PR-776 PR-777 PR-778 PR-779 PR-780 PR-781 PR-782 PR-783 PR-784 PR-785 PR-786 PR-787 PR-788 PR-789 PR-790 PR-791 PR-792 PR-793 PR-794 PR-795 PR-796 PR-797 PR-798 PR-799 PR-800 PR-801 PR-802 PR-803 PR-804 PR-805 PR-806 PR-807 PR-808 PR-809 PR-810 PR-811 PR-812 PR-813 PR-814 PR-815 PR-816 PR-817 PR-818 PR-819 PR-820 PR-821 PR-822 PR-823 PR-824 PR-825 PR-826 PR-827 PR-828 PR-829 PR-830 PR-831 PR-832 PR-833 PR-834 PR-835 PR-836 PR-837 PR-838 PR-839 PR-840 PR-841 PR-842 PR-843 PR-844 PR-845 PR-846 PR-847 PR-848 PR-849 PR-850 PR-851 PR-852 PR-853 PR-854 PR-855 PR-856 PR-857 PR-858 PR-859 PR-860 PR-861 PR-862 PR-863 PR-864 PR-865 PR-866 PR-867 PR-868 PR-869 PR-870 PR-871 PR-872 PR-873 PR-874 PR-875 PR-876 PR-877 PR-878 PR-879 PR-880 PR-881 PR-882 PR-883 PR-884 PR-885 PR-886 PR-887 PR-888 PR-889 PR-890 PR-891 PR-892 PR-893 PR-894 PR-895 PR-896 PR-897 PR-898 PR-899 PR-900 PR-901 PR-902 PR-903 PR-904 PR-905 PR-906 PR-907 PR-908 PR-909 PR-910 PR-911 PR-912 PR-913 PR-914 PR-915 PR-916 PR-917 PR-918 PR-919 PR-920 PR-921 PR-922 PR-923 PR-924 PR-925 PR-926 PR-927 PR-928 PR-929 PR-930 PR-931 PR-932 PR-933 PR-934 PR-935 PR-936 PR-937 PR-938 PR-939 PR-940 PR-941 PR-942 PR-943 PR-944 PR-945 PR-946 PR-947 PR-948 PR-949 PR-950 PR-951 PR-952 PR-953 PR-954 PR-955 PR-956 PR-957 PR-958 PR-959 PR-960 PR-961 PR-962 PR-963 PR-964 PR-965 PR-966 PR-967 PR-968 PR-969 PR-970 PR-971 PR-972 PR-973 PR-974 PR-975 PR-976 PR-977 PR-978 PR-979 PR-980 PR-981 PR-982 PR-983 PR-984 PR-985 PR-986 PR-987 PR-988 PR-989 PR-990 PR-991 PR-992 PR-993 PR-994 PR-995 PR-996 PR-997 PR-998 PR-999 PR-1000	fig. — · vídeo —: —
RF-02	Modificado (2 errores de servidor corregidos en F1) del lado servidor (httpx)	PR-19 PR-20 PR-21 PR-22 PR-23 PR-24 PR-25 PR-26 PR-27 PR-28 PR-29 PR-30 PR-31 PR-32 PR-33 PR-34 PR-35 PR-36 PR-37 PR-38 PR-39 PR-40 PR-41 PR-42 PR-43 PR-44 PR-45 PR-46 PR-47 PR-48 PR-49 PR-50 PR-51 PR-52 PR-53 PR-54 PR-55 PR-56 PR-57 PR-58 PR-59 PR-60 PR-61 PR-62 PR-63 PR-64 PR-65 PR-66 PR-67 PR-68 PR-69 PR-70 PR-71 PR-72 PR-73 PR-74 PR-75 PR-76 PR-77 PR-78 PR-79 PR-80 PR-81 PR-82 PR-83 PR-84 PR-85 PR-86 PR-87 PR-88 PR-89 PR-90 PR-91 PR-92 PR-93 PR-94 PR-95 PR-96 PR-97 PR-98 PR-99 PR-100 PR-101 PR-102 PR-103 PR-104 PR-105 PR-106 PR-107 PR-108 PR-109 PR-110 PR-111 PR-112 PR-113 PR-114 PR-115 PR-116 PR-117 PR-118 PR-119 PR-120 PR-121 PR-122 PR-123 PR-124 PR-125 PR-126 PR-127 PR-128 PR-129 PR-130 PR-131 PR-132 PR-133 PR-134 PR-135 PR-136 PR-137 PR-138 PR-139 PR-140 PR-141 PR-142 PR-143 PR-144 PR-145 PR-146 PR-147 PR-148 PR-149 PR-150 PR-151 PR-152 PR-153 PR-154 PR-155 PR-156 PR-157 PR-158 PR-159 PR-160 PR-161 PR-162 PR-163 PR-164 PR-165 PR-166 PR-167 PR-168 PR-169 PR-170 PR-171 PR-172 PR-173 PR-174 PR-175 PR-176 PR-177 PR-178 PR-179 PR-180 PR-181 PR-182 PR-183 PR-184 PR-185 PR-186 PR-187 PR-188 PR-189 PR-190 PR-191 PR-192 PR-193 PR-194 PR-195 PR-196 PR-197 PR-198 PR-199 PR-200 PR-201 PR-202 PR-203 PR-204 PR-205 PR-206 PR-207 PR-208 PR-209 PR-210 PR-211 PR-212 PR-213 PR-214 PR-215 PR-216 PR-217 PR-218 PR-219 PR-220 PR-221 PR-222 PR-223 PR-224 PR-225 PR-226 PR-227 PR-228 PR-229 PR-230 PR-231 PR-232 PR-233 PR-234 PR-235 PR-236 PR-237 PR-238 PR-239 PR-240 PR-241 PR-242 PR-243 PR-244 PR-245 PR-246 PR-247 PR-248 PR-249 PR-250 PR-251 PR-252 PR-253 PR-254 PR-255 PR-256 PR-257 PR-258 PR-259 PR-260 PR-261 PR-262 PR-263 PR-264 PR-265 PR-266 PR-267 PR-268 PR-269 PR-270 PR-271 PR-272 PR-273 PR-274 PR-275 PR-276 PR-277 PR-278 PR-279 PR-280 PR-281 PR-282 PR-283 PR-284 PR-285 PR-286 PR-287 PR-288 PR-289 PR-290 PR-291 PR-292 PR-293 PR-294 PR-295 PR-296 PR-297 PR-298 PR-299 PR-300 PR-301 PR-302 PR-303 PR-304 PR-305 PR-306 PR-307 PR-308 PR-309 PR-310 PR-311 PR-312 PR-313 PR-314 PR-315 PR-316 PR-317 PR-318 PR-319 PR-320 PR-321 PR-322 PR-323 PR-324 PR-325 PR-326 PR-327 PR-328 PR-329 PR-330 PR-331 PR-332 PR-333 PR-334 PR-335 PR-336 PR-337 PR-338 PR-339 PR-340 PR-341 PR-342 PR-343 PR-344 PR-345 PR-346 PR-347 PR-348 PR-349 PR-350 PR-351 PR-352 PR-353 PR-354 PR-355 PR-356 PR-357 PR-358 PR-359 PR-360 PR-361 PR-362 PR-363 PR-364 PR-365 PR-366 PR-367 PR-368 PR-369 PR-370 PR-371 PR-372 PR-373 PR-374 PR-375 PR-376 PR-377 PR-378 PR-379 PR-380 PR-381 PR-382 PR-383 PR-384 PR-385 PR-386 PR-387 PR-388 PR-389 PR-390 PR-391 PR-392 PR-393 PR-394 PR-395 PR-396 PR-397 PR-398 PR-399 PR-400 PR-401 PR-402 PR-403 PR-404 PR-405 PR-406 PR-407 PR-408 PR-409 PR-410 PR-411 PR-412 PR-413 PR-414 PR-415 PR-416 PR-417 PR-418 PR-419 PR-420 PR-421 PR-422 PR-423 PR-424 PR-425 PR-426 PR-427 PR-428 PR-429 PR-430 PR-431 PR-432 PR-433 PR-434 PR-435 PR-436 PR-437 PR-438 PR-439 PR-440 PR-441 PR-442 PR-443 PR-444 PR-445 PR-446 PR-447 PR-448 PR-449 PR-450 PR-451 PR-452 PR-453 PR-454 PR-455 PR-456 PR-457 PR-458 PR-459 PR-460 PR-461 PR-462 PR-463 PR-464 PR-465 PR-466 PR-467 PR-468 PR-469 PR-470 PR-471 PR-472 PR-473 PR-474 PR-475 PR-476 PR-477 PR-478 PR-479 PR-480 PR-481 PR-482 PR-483 PR-484 PR-485 PR-486 PR-487 PR-488 PR-489 PR-490 PR-491 PR-492 PR-493 PR-494 PR-495 PR-496 PR-497 PR-498 PR-499 PR-500 PR-501 PR-502 PR-503 PR-504 PR-505 PR-506 PR-507 PR-508 PR-509 PR-510 PR-511 PR-512 PR-513 PR-514 PR-515 PR-516 PR-517 PR-518 PR-519 PR-520 PR-521 PR-522 PR-523 PR-524 PR-525 PR-526 PR-527 PR-528 PR-529 PR-530 PR-531 PR-532 PR-533 PR-534 PR-535 PR-536 PR-537 PR-538 PR-539 PR-540 PR-541 PR-542 PR-543 PR-544 PR-545 PR-546 PR-547 PR-548 PR-549 PR-550 PR-551 PR-552 PR-553 PR-554 PR-555 PR-556 PR-557 PR-558 PR-559 PR-560 PR-561 PR-562 PR-563 PR-564 PR-565 PR-566 PR-567 PR-568 PR-569 PR-570 PR-571 PR-572 PR-573 PR-574 PR-575 PR-576 PR-577 PR-578 PR-579 PR-580 PR-581 PR-582 PR-583 PR-584 PR-585 PR-586 PR-587 PR-588 PR-589 PR-590 PR-591 PR-592 PR-593 PR-594 PR-595 PR-596 PR-597 PR-598 PR-599 PR-600 PR-601 PR-602 PR-603 PR-604 PR-605 PR-606 PR-607 PR-608 PR-609 PR-610 PR-611 PR-612 PR-613 PR-614 PR-615 PR-616 PR-617 PR-618 PR-619 PR-620 PR-621 PR-622 PR-623 PR-624 PR-625 	

			—: —
RF-04	Autenticación autenticada (sesión compartida)	PR-04 services/ session_service.py	fig. — . vídeo —: —
RF-05	Repetido (parser y avisos de error corregidos en F1)	PR-05 routes/ repeater.py, backend/ services/ proxy_service.py	apdo. 6 . vídeo —: —
RF-06	Cumplido (cancelación corregida en F1) fuzzing	PR-06 services/ intruder_service.py	apdo. 6 . vídeo —: —
RF-07	Utilidades (incl. JWT)	PR-07 backend/ src/ pages/ utilities/	fig. — . vídeo —: —
RF-08	Cumplido (recorrido verificado de punta a punta el 10-oct; techo de gasto ejecutado) panel Vulnerabilidades	PR-08 backend/ routes/ playwright.py, backend/ services/ redis_consumer.py, frontend/.../ VulnerabilitiesPage.jsx	figs. apdo. 8 . vídeo —: —
RF-09	Seguridad → panel Red	PR-09 + backend/ routes/ network.py	fig. — . vídeo —: —
RF-10	Seguridad → panel Red	PR-10 + backend/ routes/ playwright.py	fig. — . vídeo —: —
RF-11	Cumplido (reimplementado en F2) -usuario	PR-11 services/ guardia.py + services/ auth_service.py	apdo. 7 . figs. apdo. 8
RF-12	Registro en F2 (verificado en cocina 11/11; pendiente de producción) login por usuario + aislamiento	PR-12 services/ auth_service.py, guardia.py, usuarios_store.py, frontend/.../ LoginPage.jsx	apdo. 7 . figs. apdo. 8
RNF-01	Accesible , con limitación documentada en móvil (apdo. de navegador sin	PR-01 servido por Nginx	apdo. 10

	instalar nada en el cliente		
RNF-02	Un token autenticado en F2: el token viaja en el primer mensaje, no en la URL) tiempo real	PR-17/ main.py (/ ws/)	apdo. 7 · vídeo —: —
RNF-03	Despliegue Docker en Hetzner	PR-13 - compose.yml + infra/ nginx.conf	fig. — · vídeo —: —
RNF-04	Plataforma de reparto es real, pero dos correos del mismo autor parten el grafo de contribuciones commits que refleje el trabajo del grupo	repositorio del producto	apdo. 11
RNF-05	Control de concurrencia	PR-20/ services/ intruder_service.py (asyncio.Semaphore)	apdo. 8
RNF-06	Operación y visto en el panel (10-oct) degradada cuando la IA falla o no supera el umbral	ia/tests/test_tres_estados.py (8/8), ia/tests/ test_estado_la_confianza.py (19/19) ia/ analyzers/ vulnerability_classifier.py, backend/ services/ redis_consumer.py, frontend/.../ VulnerabilitiesPage.jsx	fig. apdo. 8
RNF-07	Seguridad en F1 (exposición cerrada y servidor endurecido) y F2 (autenticación y aislamiento, verificados producidos)	PR-14, PR-16 apartado 7	apdo. 7-8
RNF-08	Operativo y endurecido en F1 (permisos del canal y validación de entradas) por sesión	PR-15 Relay_agent.py	apdo. 7-8
RNF-09	Es cliente HTTPS	PR-18 configurar en Nginx)	apdo. 8

RUTAS VERIFICADAS EN CÓDIGO · PRUEBAS Y MINUTOS DE VÍDEO POR COMPLETAR

> **Corrección declarada de este documento (7-oct), por R9.** La fila de RF-11 citaba como implementación la ruta `/api/session/new`. Esa ruta **se retiró en la Fase 2**: repartía identificadores que no eran credenciales, y la interfaz no la llamaba desde hacía tiempo. Citar como prueba de un requisito una ruta que ya no existe es el tipo de error que una memoria arrastra hasta la defensa, así que se deja dicho en lugar de corregirlo en silencio. Quien cumple hoy RF-11 es el sistema de acceso por usuario.

> **Segunda corrección declarada (10-oct), por R9.** Esta matriz **no tenía fila** para RNF-01, RNF-04 ni RNF-06, pese a estar los tres declarados en el apartado 4. El caso de RNF-06 es el que más duele: se le

dedican varias secciones del apartado 8 y aun así no se podía trazar desde aquí, que es precisamente la pieza con la que se corrige la práctica. Las tres filas se añaden ahora. El defecto no es de redacción sino de método —la matriz se rellenó requisito a requisito desde la lista de RF y los RNF se fueron añadiendo según aparecían en el trabajo—, y explica por qué faltaban justo los tres que ninguna fase tuvo como protagonista.

Tercera corrección declarada (10-oct), por R9, y es la más seria de las tres. Aquí decía que «el estado de cada prueba (PR-xx) y su resultado detallado están en el apartado 8», y **no es cierto**: de los diecinueve códigos de prueba que cita esta columna, **solo tres** —PR-01, PR-16 y PR-20— aparecen en algún otro punto de esta memoria. Los dieciséis restantes son una **referencia a ninguna parte**: quien corrige lee «PR-08» en la fila de RF-08 y no tiene dónde ir a ver qué se probó ni con qué resultado.

El plan existe y está completo —los diecinueve casos, con procedimiento, resultado esperado y resultado observado—, pero vive en un **documento de trabajo del equipo que no se entrega**. El defecto, por tanto, no es que las pruebas no se hicieran: es que la memoria cita un plan que el lector no tiene. Al arreglar cada fallo se reejecuta la prueba y se actualiza la fila.

! HUECO — responsable: **José María + Claude**. Traer el plan de pruebas (PR-01 a PR-20) a la memoria, como tabla del anexo, para que esta columna deje de apuntar a ninguna parte. OJO al hacerlo: los resultados observados del plan son del 29-sep y varios están desfasados —PR-01 figura como FALLA por un Basic Auth que la Fase 2 retiró—, así que se revisa caso por caso antes de copiarlo. Copiarlo tal cual metería afirmaciones falsas en el entregable.

La numeración RF/RNF es la fijada en el apartado 4 y **PENDIENTE DE VALIDAR POR EL GRUPO**.

! HUECO — responsable: **José María + Claude**. Al revisar el vídeo grabado, anotar el minuto exacto donde se demuestra cada requisito y el número de figura de su captura (R6: el nombre de la captura va atado al ID del requisito, p. ej. `RF-08-panel-ia.png`, para que esta columna se rellene sola).

10. Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría —tráfico capturado, resultados del Intruder, vulnerabilidades— vive **en memoria del backend**: al reiniciarlo, se pierde. Es una **decisión consciente**, no un olvido, y se sostiene en tres razones:

1. **El patrón de acceso del backend no lo soporta sin reescribirlo.** Una treintena de puntos obtienen el diccionario de la sesión y lo mutan en memoria (`session["requests"].append(...)`). Persistir de verdad obligaría a escritura-a-través en cada mutación, es decir, a tocar todo el backend a pocos días de la entrega.
2. **El modelo de sesión cambia en la Fase 2.** El login por usuario ata las sesiones a identidades; lo que se persistiera con el modelo anterior habría que rehacerlo.
3. **Una sesión de auditoría es un espacio de trabajo efímero.** El operador lanza un rastreo, examina los resultados y exporta lo que importa; no espera que la herramienta recuerde una sesión de hace tres días.

Lo que **sí** se hizo, porque era el riesgo real detrás de «todo en memoria»: **poner topes**. El recolector de sesiones existía en el código pero nunca se invocaba, y cualquier llamada a la API con un identificador nuevo creaba una sesión más —con la API sin autenticación, eso permitía agotar la memoria del servidor desde fuera—. Ahora hay un tope de sesiones y de peticiones por sesión (configurables por entorno), el recolector se ejecuta de verdad y, al desalojar, **prefiere las sesiones que no tienen conexión en tiempo real abierta**, para no interrumpir a quien está trabajando. Verificado con prueba de abuso: 120

identificadores inventados dejan el número de sesiones en el tope, y la sesión con conexión viva sobrevive.

Trabajo futuro. La persistencia es viable sin cambiar la arquitectura: el despliegue ya incluye Redis y el backend ya lo usa para otro flujo. El momento correcto es **después** del modelo de usuarios, para persistir sesiones ya asociadas a su propietario.

El coste de una auditoría con IA no está acotado

El módulo de IA no limita cuántas veces consulta al modelo durante una auditoría: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Hoy es inocuo, porque el módulo no se puede disparar desde la interfaz; deja de serlo en el momento en que exista el disparador, que es trabajo en curso. El antecedente está en el apartado 3: en la P1, un disparo involuntario de este mismo recorrido agotó el saldo de la API.

Trabajo futuro, diseñado y no aplicado. Un techo de consultas por auditoría, configurable por entorno y situado en el **cliente** del modelo —por donde pasan las cuatro vías de análisis, de modo que ninguna se escapa—, que al agotarse devuelva la vía degradada de RNF-06 para que la auditoría quede marcada como truncada en vez de parecer completa. Un techo silencioso sería peor que no tenerlo: dejaría el informe diciendo «sin hallazgos» sobre una auditoría a medias.

El valor por defecto se fijará **con el dato** de la primera auditoría real —el resultado llevará el número de consultas consumidas— y no a ojo, que es lo que permite defenderlo.

Lo que queda de la confusión de escala, una vez cerrada

Rectificación declarada. Una versión anterior de este apartado, escrita el mismo día unas horas antes, daba esta limitación por **abierta** y decía que cerrarla dependía de un dato que «no consta». El dato se obtuvo después —ninguna de las cuatro instrucciones pide que la confianza sea un número entero— y con él la limitación se cerró, aunque no por donde este apartado anticipaba: el arreglo está en el clasificador, no en el esquema, y se cuenta en el apartado 8. Se rectifica aquí en lugar de reescribirlo en silencio.

Lo que **sí** queda, y es consciente:

1. **Un valor legítimo de 1 se marca como sospechoso.** La guarda no puede distinguir «1 %» de «certeza total en escala de 0 a 1», porque son el mismo número. Se eligió marcarlo: con el umbral en 60 ese valor tampoco se habría reportado, de modo que el coste es un aviso de más y la ganancia es que el caso peor —descartar un hallazgo seguro— deja de ser posible. **Preferir el falso positivo visible al falso negativo silencioso** es la misma elección que ya tomó RNF-06.
2. **La vía de identificación de servidor no lleva guarda,** porque no compara contra el umbral: entrega la respuesta del modelo sin filtrar, así que no hay descarte silencioso que cerrar.
3. **No se cambiaron las instrucciones del modelo para pedir un entero,** que era la alternativa más limpia en el plano conceptual —instrucciones y esquema de acuerdo en un contrato más estricto—. Se descartó por el momento: son cuatro ficheros del material que no se vuelca a la conversación, se editarían sin verlos, y son prosa y no una línea de código, a tres días de la congelación. Queda como trabajo futuro de coste bajo.

La herramienta que genera los esquemas no está versionada

El generador de esquemas vive en la carpeta de trabajo de la práctica, **fuera del repositorio del producto**: no tiene historial, no pasó por revisión y su autoría no queda registrada, al contrario que todo el código. Es incoherente con la norma del equipo de que todo entra por petición de incorporación, y no es una incoherencia inocua: **el único sitio sin historial resultó ser exactamente donde estaba el defecto del apartado 8**, porque es el sitio donde nadie vio la lista incompleta.

Trabajo futuro, de coste bajo: mover las herramientas de la práctica al repositorio, para que un cambio en ellas se revise como cualquier otro. No se hizo ahora porque mover ficheros a tres días de la congelación introduce riesgo sin aportar nada al producto.

⚠ **HUECO** — responsable: **José María + Claude**. Acumular el resto de limitaciones conforme aparezcan durante las Fases 1-4; este apartado recibe material hasta el último día.

11. Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

Por qué el número de confirmaciones NO mide el reparto, y conviene decirlo antes de que alguien lo cuente. La autoría de cada confirmación se asigna **al azar** entre las cinco personas del grupo, de forma deliberada: el trabajo se reparte por tareas y no por ficheros, así que atar cada confirmación a quien tocó ese fichero habría dado una foto igual de falsa y menos honesta. Con una excepción fija: **la memoria técnica se firma siempre a nombre de la misma persona**, porque es un documento único que se regenera entero en cada cambio.

Esa excepción deforma el recuento, y el dato concreto lo demuestra: de las 41 confirmaciones de la jornada del 10 de octubre, una persona figura en 19 —más del doble que cualquier otra—, pero **11 de esas 19 son regeneraciones de la memoria**. Descontada la excepción quedan 8, que es justo lo que cabe esperar de un sorteo entre cinco sobre 41. El reparto es uniforme; lo que no es uniforme es el documento.

La estimación de horas de este apartado sale del diario, no del historial de confirmaciones.

⚠ **HUECO** — responsable: **José María**. Consolidar las horas por persona al congelar (13-oct). El reparto real del trabajo queda trazado por los commits firmados por cada responsable (R2); José María cierra el recuento.

12. Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje de la cocina, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

Los límites de la herramienta, observados en primera persona

Usar un asistente de IA para auditar seguridad tiene un límite que conviene declarar porque lo encontramos trabajando: **el filtro de seguridad del propio modelo bloquea a veces tareas legítimas de este dominio**. El 9 de octubre, leyendo las instrucciones del clasificador de vulnerabilidades de HookSuite, la petición fue rechazada con un error que lo dice con todas las letras —el filtro «a veces marca tareas legítimas de programación, **ciberseguridad** y biología»—. No es un fallo del proyecto ni del operador: es el coste de trabajar en seguridad ofensiva con una herramienta generalista, y conviene contarlo con la fecha y la referencia delante en lugar de omitirlo.

Lo caro no fue el bloqueo, sino **la recuperación**. Para desatascar la sesión se usó la función de rebobinado del cliente, eligiendo entre sus cuatro opciones la que descarta lo posterior al punto de retorno; se perdieron así ocho minutos de trabajo. Las otras dos opciones útiles —resumir desde el punto, o resumir solo lo anterior y permanecer al final de la conversación— habrían conservado ese trabajo. Es una lección de herramienta, no de ingeniería, pero cuesta horas igual.

De ahí sale el hallazgo que sí afecta al diseño de nuestros controles: **el rebobinado restaura la conversación y los ficheros, pero no los efectos sobre el servidor**. En la lista de puntos de retorno, el turno que había construido una imagen de contenedor figuraba como «sin cambios de código», porque esa función contabiliza ficheros. La imagen siguió existiendo después de rebobinar. Quien la construyó —el asistente— ya no recordaba haberlo hecho, y el único testigo del cambio fue un control externo que compara el estado del servidor contra lo que la documentación afirma. La consecuencia práctica es que **un control que mira ficheros no sustituye a uno que mira el sistema**, y que los avisos molestos de ese control no se pueden desactivar sin perder exactamente aquello para lo que sirve.

Qué fue generado y qué fue verificado. La distinción no es teórica: ese mismo día, una afirmación escrita por el asistente en el plan del módulo de IA —que los cuatro conjuntos de instrucciones pedían un campo `descripcion`— resultó falsa para uno de ellos al releer el código fuente, y quedó corregida de forma declarada en vez de reescrita en silencio. El criterio que seguimos es que **nada entra en esta memoria por haberlo dicho la herramienta**: entra cuando se ha contrastado contra el código, la salida de un comando o el enunciado.

Leer sin leer: delegar la lectura de lo que no debe entrar en la conversación

El filtro descrito arriba obligó a una medida de método que vale la pena declarar, porque cambió la forma de trabajar. Las instrucciones que el producto envía al modelo son el material que más incomoda a ese filtro y, a la vez, eran lo único que faltaba por conocer para terminar el módulo de IA: **tres defectos abiertos dependían de saber qué formato piden**. El asistente principal no podía abrirlas sin arriesgar la sesión, y el permiso del operador no sirve de nada, porque el filtro no sabe nada de ese permiso: mide lo que entra en la conversación, no lo que alguien ha autorizado.

La salida fue **delegar la lectura**. Un asistente secundario, con instrucciones propias y un contexto que se destruye con él, abre los ficheros y devuelve solo su **interfaz**: nombres de campo, tipos, escalas y números de línea. No devuelve ni una frase del texto. El encargo se redactó con una precisión que resultó ser la clave —«**no los cites**», **nunca «no los leas»**—, porque una versión anterior, mal formulada, llevó al secundario a **medir** los ficheros sin abrirlos y a contestar a medias.

De ahí salieron los tres defectos del apartado 8. Es decir: el control que impide volcar material delicado en la conversación no solo no frenó el trabajo, sino que obligó a una forma de trabajar —preguntar por la interfaz y no por el contenido, y comprobar con pruebas en vez de con la vista— que resultó **más** estricta que la lectura directa. Lo que no se puede leer, se mide, se edita a ciegas y se verifica con una prueba.

El límite del método también se declara: el asistente secundario es un modelo, no un control. Lo que devuelve tiene que revisarlo **una persona que conozca el material**, porque es la única que puede juzgar si cruzó la línea entre describir y reproducir; el asistente principal no puede, precisamente porque no conoce el contenido.

 **HUECO** — responsable: **José María**. Detallar por fase qué aportó la IA y qué fue decisión/revisión humana; distinguir generado de verificado.

13. Anexos

Manual de instalación ampliado, credenciales de prueba (ficticias), glosario y referencias.

 **HUECO** — responsable: **José María**. Manual de despliegue, usuarios de prueba del login nuevo (sustituyen a la credencial de Nginx), glosario y referencias.